

力扣hot 100总结

哈希

两数之和

```

1  class Solution {
2  public:
3      vector<int> twoSum(vector<int>& nums, int target) {
4          unordered_map<int,int> nmap;
5
6          for(int i=0; i<nums.size(); ++i){
7              auto iter = nmap.find(target - nums[i]);
8              if (iter != nmap.end()){
9                  return {iter->second,i};
10             }
11             nmap.insert(pair<int,int>(nums[i],i));
12         }
13
14         return {};
15     }
16 };

```

字母异位词分组

```

1  class Solution {
2  public:
3      vector<vector<string>> groupAnagrams(vector<string>& strs) {
4          unordered_map<string,vector<string>> mp;
5          vector<vector<string>> result;
6
7          for(auto& str:strs){
8              string key = str;
9              sort(key.begin(),key.end());
10             mp[key].push_back(str);
11         }
12
13         for(auto it=mp.begin(); it!=mp.end(); ++it){
14             result.push_back(it->second);
15         }
16
17         return result;
18     }
19 };

```

最长连续序列

```

1  class Solution {
2  public:
3      int longestConsecutive(vector<int>& nums) {
4          unordered_set<int> num_set;

```

```

5         int maxLength = 0;
6         for(auto& num:nums){
7             num_set.insert(num);
8         }
9
10        for(auto& num:num_set){//注意不要遍历数组，遍历集合
11            if(!num_set.count(num-1)){
12                int cur_num = num;
13                int cur_length = 1;
14
15                while(num_set.count(cur_num+1)){
16                    cur_num++;
17                    cur_length++;
18                }
19
20                maxLength = max(maxLength,cur_length);
21            }
22        }
23
24        return maxLength;
25    }
26 };

```

双指针

移动零

```

1  class Solution {
2  public:
3      void moveZeroes(vector<int>& nums) {
4          int l=0,r=0;
5          while(r<nums.size()){
6              if(nums[r] != 0){
7                  swap(nums[l],nums[r]);
8                  l++;
9              }
10             r++;
11         }
12     }
13 };

```

盛水最多的容器

```

1  class Solution {
2  public:
3      int maxArea(vector<int>& height) {
4          int l = 0, r = height.size() - 1;
5          int ans = 0;
6          while (l < r) {
7              int area = min(height[l], height[r]) * (r - l);
8              ans = max(ans, area);
9              if (height[l] <= height[r]) {
10                 ++l;

```

```

11         }
12         else {
13             --r;
14         }
15     }
16     return ans;
17 }
18 };

```

三数之和

```

1  class Solution {
2  public:
3      vector<vector<int>> threeSum(vector<int>& nums) {
4          vector<vector<int>> result;
5          sort(nums.begin(), nums.end());
6
7          for (int i = 0; i < nums.size(); i++) {
8              if (nums[i] > 0) {
9                  return result;
10             }
11             // 正确去重a方法
12             if (i > 0 && nums[i] == nums[i - 1]) {
13                 continue;
14             }
15             int left = i + 1;
16             int right = nums.size() - 1;
17             while (right > left) {
18                 if (nums[i] + nums[left] + nums[right] > 0) right--;
19                 else if (nums[i] + nums[left] + nums[right] < 0) left++;
20                 else {
21                     result.push_back(vector<int>{nums[i], nums[left],
22 nums[right]});
23                     // 去重逻辑应该放在找到一个三元组之后, 对b 和 c去重
24                     while (right > left && nums[right] == nums[right - 1])
25                         right--;
26                     while (right > left && nums[left] == nums[left + 1])
27                         left++;
28                     right--;
29                     left++;
30                 }
31             }
32             }
33         return result;
34     }
35 };

```

接雨水

```

1  class Solution {
2  public:
3      int trap(vector<int>& height) {
4          stack<int> st; // 存着下标
5          st.push(0);

```

```

6         int sum = 0;
7         for (int i = 1; i < height.size(); i++) {
8             if (height[i] < height[st.top()]) { // 情况一
9                 st.push(i);
10            } if (height[i] == height[st.top()]) { // 情况二
11                st.pop();
12                st.push(i);
13            } else { // 情况三
14                while (!st.empty() && height[i] > height[st.top()]) {
15                    int mid = st.top();
16                    st.pop();
17                    if (!st.empty()) {
18                        int h = min(height[st.top()], height[i]) -
height[mid];
19                        int w = i - st.top() - 1; // 注意减一，只求中间宽度
20                        sum += h * w;
21                    }
22                }
23                st.push(i);
24            }
25        }
26        return sum;
27    }
28 };

```

滑动窗口

无重复字符的最长子串

```

1     class Solution {
2     public:
3         int lengthOfLongestSubstring(string s) {
4             unordered_set<char> vocab;
5             int cur_length = 0;
6             int result = 0;
7             int left = 0 ;
8
9             for(int right = 0;right<s.size();++right){
10                auto ch = s[right];
11
12                while(vocab.count(ch)){
13                    vocab.erase(s[left]);
14                    left++;
15                }
16
17                vocab.insert(ch);
18                cur_length = right-left+1;
19
20                result = max(result,cur_length);
21            }
22            return result;
23        }
24    };

```

找到字符串中所有字母异位词

```

1  class Solution {
2  public:
3      vector<int> findAnagrams(string s, string p) {
4          int sLen = s.size(), pLen = p.size();
5
6          if (sLen < pLen) {
7              return vector<int>();
8          }
9          vector<int> ans;
10         vector<int> sCount(26);
11         vector<int> pCount(26);
12         for (int i = 0; i < pLen; ++i) {
13             ++sCount[s[i] - 'a'];
14             ++pCount[p[i] - 'a'];
15         }
16         if (sCount == pCount) {
17             ans.emplace_back(0);
18         }
19         for (int i = 0; i < sLen - pLen; ++i) {
20             --sCount[s[i] - 'a'];
21             ++sCount[s[i + pLen] - 'a'];
22
23             if (sCount == pCount) {
24                 ans.emplace_back(i + 1);
25             }
26         }
27         return ans;
28     }
29 };

```

子串

和为k的连续子数组

```

1  class Solution {
2  public:
3      int subarraySum(vector<int>& nums, int k) {
4          unordered_map<int, int> mp;
5          mp[0] = 1;
6          int count = 0, pre=0;
7          for(int i = 0; i<nums.size();++i){
8              pre+=nums[i];
9              if(mp.find(pre-k)!=mp.end()){
10                 count+=mp[pre-k];
11             }
12             mp[pre]++;
13         }
14         return count;
15     }
16 };

```

滑动窗口最大值

```

1  class Solution {
2  public:
3      vector<int> maxSlidingWindow(vector<int>& nums, int k) {
4          int n = nums.size();
5          priority_queue<pair<int, int>> q;
6          for (int i = 0; i < k; ++i) {
7              q.emplace(nums[i], i);
8          }
9          vector<int> ans = {q.top().first};
10         for (int i = k; i < n; ++i) {
11             q.emplace(nums[i], i);
12             while (q.top().second <= i - k) {
13                 q.pop();
14             }
15             ans.push_back(q.top().first);
16         }
17         return ans;
18     }
19 };

```

最小覆盖子串

```

1  class Solution {
2  public:
3      unordered_map<char, int> ori, cnt;
4
5      bool check() {
6          for (const auto &p: ori) {
7              if (cnt[p.first] < p.second) {
8                  return false;
9              }
10         }
11         return true;
12     }
13
14     string minWindow(string s, string t) {
15         for (const auto &c: t) {
16             ++ori[c];
17         }
18
19         int l = 0, r = -1;
20         int len = INT_MAX, ansL = -1, ansR = -1;
21
22         while (r < int(s.size())) {
23             if (ori.find(s[++r]) != ori.end()) {
24                 ++cnt[s[r]];
25             }
26             while (check() && l <= r) {
27                 if (r - l + 1 < len) {
28                     len = r - l + 1;
29                     ansL = l;
30                 }

```

```

31         if (ori.find(s[l]) != ori.end()) {
32             --cnt[s[l]];
33         }
34         ++l;
35     }
36 }
37
38 return ansL == -1 ? string() : s.substr(ansL, len);
39 }
40 };

```

普通数组

最大子数组和

```

1  class Solution {
2  public:
3      int maxSubArray(vector<int>& nums) {
4          int pre = 0, maxAns = nums[0];
5          for (const auto &x: nums) {
6              pre = max(pre + x, x);
7              maxAns = max(maxAns, pre);
8          }
9          return maxAns;
10     }
11 };

```

合并区间

```

1  class Solution {
2  public:
3      vector<vector<int>> merge(vector<vector<int>>& intervals) {
4          vector<vector<int>> result;
5          if (intervals.size() == 0) return result; // 区间集合为空直接返回
6          // 排序的参数使用了lambda表达式
7          sort(intervals.begin(), intervals.end());
8          // [](const vector<int>& a, const vector<int>& b){return a[0] <
b[0];}
9
10         // 第一个区间就可以放进结果集里，后面如果重叠，在result上直接合并
11         result.push_back(intervals[0]);
12
13         for (int i = 1; i < intervals.size(); i++) {
14             if (result.back()[1] >= intervals[i][0]) { // 发现重叠区间
15                 // 合并区间，只更新右边界就好，因为result.back()的左边界一定是最小值，
因为我们按照左边界排序的
16                 result.back()[1] = max(result.back()[1], intervals[i][1]);
17             } else {
18                 result.push_back(intervals[i]); // 区间不重叠
19             }
20         }
21         return result;
22     }

```

23 };

轮转数组

```

1  class Solution {
2  public:
3      void reverse(vector<int>& nums, int start, int end) {
4          while (start < end) {
5              swap(nums[start], nums[end]);
6              start += 1;
7              end -= 1;
8          }
9      }
10
11     void rotate(vector<int>& nums, int k) {
12         k %= nums.size(); //注意这行处理
13         reverse(nums, 0, nums.size() - 1);
14         reverse(nums, 0, k - 1);
15         reverse(nums, k, nums.size() - 1);
16     }
17 };

```

除自身以外乘积

```

1  class Solution {
2  public:
3      vector<int> productExceptSelf(vector<int>& nums) {
4          int length = nums.size();
5
6          vector<int> L(length, 0), R(length, 0);
7          vector<int> answer(length);
8
9          L[0] = 1;
10         for (int i = 1; i < length; i++) {
11             L[i] = nums[i - 1] * L[i - 1];
12         }
13
14         R[length - 1] = 1;
15         for (int i = length - 2; i >= 0; i--) {
16             R[i] = nums[i + 1] * R[i + 1];
17         }
18
19         for (int i = 0; i < length; i++) {
20             answer[i] = L[i] * R[i];
21         }
22
23         return answer;
24     }
25 };

```

缺失的第一个正数

```

1  class Solution {

```

```

2 public:
3     int firstMissingPositive(vector<int>& nums) {
4         int n = nums.size();
5         for (int& num: nums) {
6             if (num <= 0) {
7                 num = n + 1;
8             }
9         }
10        for (int i = 0; i < n; ++i) {
11            int num = abs(nums[i]);
12            if (num <= n) {
13                nums[num - 1] = -abs(nums[num - 1]);
14            }
15        }
16        for (int i = 0; i < n; ++i) {
17            if (nums[i] > 0) {
18                return i + 1;
19            }
20        }
21        return n + 1;
22    }
23 };

```

矩阵

矩阵置0

```

1 class Solution {
2 public:
3     void setZeroes(vector<vector<int>>& matrix) {
4         int m = matrix.size();
5         int n = matrix[0].size();
6         vector<int> row(m), col(n);
7         for (int i = 0; i < m; i++) {
8             for (int j = 0; j < n; j++) {
9                 if (!matrix[i][j]) {
10                    row[i] = col[j] = true;
11                }
12            }
13        }
14        for (int i = 0; i < m; i++) {
15            for (int j = 0; j < n; j++) {
16                if (row[i] || col[j]) {
17                    matrix[i][j] = 0;
18                }
19            }
20        }
21    }
22 };

```

螺旋矩阵

```

1 class Solution {

```

```

2 private:
3     static constexpr int directions[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1,
0}}};
4 public:
5     vector<int> spiralOrder(vector<vector<int>>& matrix) {
6         if (matrix.size() == 0 || matrix[0].size() == 0) {
7             return {};
8         }
9
10        int rows = matrix.size(), columns = matrix[0].size();
11        vector<vector<bool>> visited(rows, vector<bool>(columns));
12        int total = rows * columns;
13        vector<int> order(total);
14
15        int row = 0, column = 0;
16        int directionIndex = 0;
17        for (int i = 0; i < total; i++) {
18            order[i] = matrix[row][column];
19            visited[row][column] = true;
20            int nextRow = row + directions[directionIndex][0], nextColumn =
column + directions[directionIndex][1];
21            if (nextRow < 0 || nextRow >= rows || nextColumn < 0 ||
nextColumn >= columns || visited[nextRow][nextColumn]) {
22                directionIndex = (directionIndex + 1) % 4;
23            }
24            row += directions[directionIndex][0];
25            column += directions[directionIndex][1];
26        }
27        return order;
28    }
29 };

```

旋转图像

```

1 class Solution {
2 public:
3     void rotate(vector<vector<int>>& matrix) {
4         int n = matrix.size();
5         // C++ 这里的 = 拷贝是值拷贝，会得到一个新的数组
6         auto matrix_new = matrix;
7         for (int i = 0; i < n; ++i) {
8             for (int j = 0; j < n; ++j) {
9                 matrix_new[j][n - i - 1] = matrix[i][j];
10            }
11        }
12        // 这里也是值拷贝
13        matrix = matrix_new;
14    }
15 };

```

搜索二维矩阵II

```

1  class Solution {
2  public:
3      bool searchMatrix(vector<vector<int>>& matrix, int target) {
4          for (const auto& row: matrix) {
5              auto it = lower_bound(row.begin(), row.end(), target);
6              if (it != row.end() && *it == target) {
7                  return true;
8              }
9          }
10         return false;
11     }
12 };

```

链表

相交链表

```

1  class Solution {
2  public:
3      ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
4          ListNode *A = headA, *B = headB;
5          while (A != B) {
6              A = A != nullptr ? A->next : headB;
7              B = B != nullptr ? B->next : headA;
8          }
9          return A;
10     }
11 };

```

反转链表

```

1  class Solution {
2  public:
3      ListNode* reverseList(ListNode* head) {
4          ListNode* pre=nullptr;
5          ListNode* cur=head;
6          ListNode* temp=nullptr;
7          while(cur){
8              temp = cur -> next;
9              cur -> next = pre;
10             pre = cur;
11             cur = temp;
12         }
13         return pre;
14     }
15 };

```

回文链表

```

1  class Solution {
2  public:
3      bool isPalindrome(ListNode* head) {
4          vector<int> vals;
5          while (head != nullptr) {
6              vals.emplace_back(head->val);
7              head = head->next;
8          }
9          for (int i = 0, j = (int)vals.size() - 1; i < j; ++i, --j) {
10             if (vals[i] != vals[j]) {
11                 return false;
12             }
13         }
14         return true;
15     }
16 };

```

环形链表

```

1  class Solution {
2  public:
3      bool hasCycle(ListNode *head) {
4          ListNode *slow = head;
5          ListNode *fast = head;
6          while(fast != nullptr) {
7              fast = fast->next;
8              if(fast != nullptr) {
9                  fast = fast->next;
10             }
11             if(fast == slow) {
12                 return true;
13             }
14             slow = slow->next;
15         }
16         return false;
17     }
18 };
19

```

环形链表II

```

1  class Solution {
2  public:
3      ListNode *detectCycle(ListNode *head) {
4          ListNode* slow = head;
5          ListNode* fast = head;
6          while(fast != NULL && fast->next != NULL){
7              slow = slow -> next;
8              fast = fast -> next -> next;
9              if(slow == fast){
10                 ListNode* index1 = fast;
11                 ListNode* index2 = head;
12                 while(index1 != index2){
13                     index1 = index1->next;

```

```

14         index2 = index2->next;
15     }
16     return index2;
17 }
18 }
19 return NULL;
20 }
21 };

```

合并两个有序链表

```

1  class Solution {
2  public:
3      ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
4          ListNode* preHead = new ListNode(-1);
5
6          ListNode* prev = preHead;
7          while (l1 != nullptr && l2 != nullptr) {
8              if (l1->val < l2->val) {
9                  prev->next = l1;
10                 l1 = l1->next;
11             } else {
12                 prev->next = l2;
13                 l2 = l2->next;
14             }
15             prev = prev->next;
16         }
17         // 合并后 l1 和 l2 最多只有一个还未被合并完，我们直接将链表末尾指向未合并完的链
表即可
18         prev->next = l1 == nullptr ? l2 : l1;
19
20         return preHead->next;
21     }
22 };

```

两数相加

```

1  class Solution {
2  public:
3      ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
4          ListNode *head = nullptr, *tail = nullptr;
5          int carry = 0;
6          while (l1 || l2) {
7              int n1 = l1 ? l1->val : 0;
8              int n2 = l2 ? l2->val : 0;
9              int sum = n1 + n2 + carry;
10             if (!head) {
11                 head = tail = new ListNode(sum % 10);
12             } else {
13                 tail->next = new ListNode(sum % 10);
14                 tail = tail->next;
15             }
16             carry = sum / 10;
17             if (l1) {

```

```

18         l1 = l1->next;
19     }
20     if (l2) {
21         l2 = l2->next;
22     }
23 }
24 if (carry > 0) {
25     tail->next = new ListNode(carry);
26 }
27 return head;
28 }
29 };

```

删除链表倒数第N个结点

```

1  class Solution {
2  public:
3      ListNode* removeNthFromEnd(ListNode* head, int n) {
4          ListNode* dummyhead = new ListNode(0);
5          dummyhead->next = head;
6          ListNode* slow = dummyhead;
7          ListNode* fast = dummyhead;
8          while(n-- && fast!=NULL){
9              fast = fast->next;
10         }
11         fast = fast -> next;
12         while(fast != NULL){
13             fast = fast->next;
14             slow = slow->next;
15         }
16         ListNode* tmp = slow->next;
17         slow->next = tmp -> next;
18         delete tmp;
19
20         return dummyhead->next;
21     }
22 };

```

两两交换链表中的节点

```

1  class Solution {
2  public:
3      ListNode* swapPairs(ListNode* head) {
4          ListNode* dummyhead = new ListNode(0);
5          dummyhead->next = head;
6          ListNode* cur = dummyhead;
7          while(cur->next!=nullptr && cur->next->next!=nullptr){
8              ListNode* tmp = cur->next;
9              ListNode* tmp1 = cur->next->next->next;
10
11              cur->next = cur->next->next;
12              cur->next->next = tmp;
13              cur->next->next->next = tmp1;
14

```

```

15         cur = cur->next->next;
16     }
17     ListNode* result = dummyhead->next;
18     delete dummyhead;
19     return result;
20 }
21 };

```

K个一组翻转链表

```

1  class Solution {
2  public:
3      // 翻转一个子链表，并且返回新的头与尾
4      pair<ListNode*, ListNode*> myReverse(ListNode* head, ListNode* tail) {
5          ListNode* prev = tail->next;
6          ListNode* p = head;
7          while (prev != tail) {
8              ListNode* nex = p->next;
9              p->next = prev;
10             prev = p;
11             p = nex;
12         }
13         return {tail, head};
14     }
15
16     ListNode* reverseKGroup(ListNode* head, int k) {
17         ListNode* hair = new ListNode(0);
18         hair->next = head;
19         ListNode* pre = hair;
20
21         while (head) {
22             ListNode* tail = pre;
23             // 查看剩余部分长度是否大于等于 k
24             for (int i = 0; i < k; ++i) {
25                 tail = tail->next;
26                 if (!tail) {
27                     return hair->next;
28                 }
29             }
30             ListNode* nex = tail->next;
31             // 这里是 C++17 的写法，也可以写成
32             // pair<ListNode*, ListNode*> result = myReverse(head, tail);
33             // head = result.first;
34             // tail = result.second;
35             tie(head, tail) = myReverse(head, tail);
36             // 把子链表重新接回原链表
37             pre->next = head;
38             tail->next = nex;
39             pre = tail;
40             head = tail->next;
41         }
42
43         return hair->next;
44     }

```

45 };

排序链表

```

1  class Solution {
2  public:
3      ListNode* sortList(ListNode* head) {
4          return sortList(head, nullptr);
5      }
6
7      ListNode* sortList(ListNode* head, ListNode* tail) {
8          if (head == nullptr) {
9              return head;
10         }
11         if (head->next == tail) {
12             head->next = nullptr;
13             return head;
14         }
15         ListNode* slow = head, *fast = head;
16         while (fast != tail) {
17             slow = slow->next;
18             fast = fast->next;
19             if (fast != tail) {
20                 fast = fast->next;
21             }
22         }
23         ListNode* mid = slow;
24         return merge(sortList(head, mid), sortList(mid, tail));
25     }
26
27     ListNode* merge(ListNode* head1, ListNode* head2) {
28         ListNode* dummyHead = new ListNode(0);
29         ListNode* temp = dummyHead, *temp1 = head1, *temp2 = head2;
30         while (temp1 != nullptr && temp2 != nullptr) {
31             if (temp1->val <= temp2->val) {
32                 temp->next = temp1;
33                 temp1 = temp1->next;
34             } else {
35                 temp->next = temp2;
36                 temp2 = temp2->next;
37             }
38             temp = temp->next;
39         }
40         if (temp1 != nullptr) {
41             temp->next = temp1;
42         } else if (temp2 != nullptr) {
43             temp->next = temp2;
44         }
45         return dummyHead->next;
46     }
47 };

```

合并k个升序链表

```

1 class Solution {
2 public:
3     ListNode* mergeTwoLists(ListNode *a, ListNode *b) {
4         if ((!a) || (!b)) return a ? a : b;
5         ListNode head, *tail = &head, *aPtr = a, *bPtr = b;
6         while (aPtr && bPtr) {
7             if (aPtr->val < bPtr->val) {
8                 tail->next = aPtr; aPtr = aPtr->next;
9             } else {
10                 tail->next = bPtr; bPtr = bPtr->next;
11             }
12             tail = tail->next;
13         }
14         tail->next = (aPtr ? aPtr : bPtr);
15         return head.next;
16     }
17
18     ListNode* mergeKLists(vector<ListNode*>& lists) {
19         ListNode *ans = nullptr;
20         for (size_t i = 0; i < lists.size(); ++i) {
21             ans = mergeTwoLists(ans, lists[i]);
22         }
23         return ans;
24     }
25 };

```

二叉树

中序遍历

```

1 class Solution {
2 public:
3     vector<int> result;
4     void traversal(TreeNode* root){
5         if(root == NULL) return;
6
7         traversal(root->left);
8         result.push_back(root->val);
9         traversal(root->right);
10    }
11
12
13    vector<int> inorderTraversal(TreeNode* root) {
14        traversal(root);
15        return result;
16    }
17 };

```

二叉树的最大深度

```

1 class Solution {

```

```

2 public:
3     int maxDepth(TreeNode* root) {
4         if (root == nullptr) return 0;
5         queue<TreeNode*> Q;
6         Q.push(root);
7         int ans = 0;
8         while (!Q.empty()) {
9             int sz = Q.size();
10            while (sz > 0) {
11                TreeNode* node = Q.front(); Q.pop();
12                if (node->left) Q.push(node->left);
13                if (node->right) Q.push(node->right);
14                sz --;
15            }
16            ans ++;
17        }
18        return ans;
19    }
20 };

```

翻转二叉树

```

1 class Solution {
2 public:
3     TreeNode* invertTree(TreeNode* root) {
4         std::stack<TreeNode*> st;
5         if (root != NULL) st.push(root);
6         while (!st.empty()) {
7             TreeNode* node = st.top();
8             st.pop();
9             swap(node->left, node->right);
10            if (node->right) st.push(node->right);
11            if (node->left) st.push(node->left);
12        }
13        return root;
14    }
15 };

```

对称二叉树

```

1 class Solution {
2 public:
3     bool isSymmetric(TreeNode* root) {
4         if (root == NULL) return true;
5         std::queue<TreeNode*> que;
6         que.push(root->left);
7         que.push(root->right);
8
9         while (!que.empty()) {
10            TreeNode* leftnode = que.front(); que.pop();
11            TreeNode* rightnode = que.front(); que.pop();
12            if (!leftnode && !rightnode) continue;
13

```

```

14         if(!leftnode || !rightnode || leftnode->val != rightnode->val)
            return false;
15
16         que.push(leftnode->left);
17         que.push(rightnode->right);
18         que.push(leftnode->right);
19         que.push(rightnode->left);
20     }
21     return true;
22
23 }
24 };

```

二叉树的直径

```

1  class Solution {
2      int ans;
3      int depth(TreeNode* rt){
4          if (rt == NULL) {
5              return 0; // 访问到空节点了，返回0
6          }
7          int L = depth(rt->left); // 左儿子为根的子树的深度
8          int R = depth(rt->right); // 右儿子为根的子树的深度
9          ans = max(ans, L + R + 1); // 计算d_node即L+R+1 并更新ans
10         return max(L, R) + 1; // 返回该节点为根的子树的深度
11     }
12 public:
13     int diameterOfBinaryTree(TreeNode* root) {
14         depth(root);
15         return ans - 1;
16     }
17 };

```

二叉树的层序遍历

```

1  class Solution {
2  public:
3      vector<vector<int>> levelOrder(TreeNode* root) {
4          queue<TreeNode*> que;
5          if (root != NULL) que.push(root);
6          vector<vector<int>> result;
7          while (!que.empty()) {
8              int size = que.size();
9              vector<int> vec;
10             // 这里一定要使用固定大小size，不要使用que.size()，因为que.size是不断变化
               的
11             for (int i = 0; i < size; i++) {
12                 TreeNode* node = que.front();
13                 que.pop();
14                 vec.push_back(node->val);
15                 if (node->left) que.push(node->left);
16                 if (node->right) que.push(node->right);
17             }
18             result.push_back(vec);

```

```

19     }
20     return result;
21 }
22 };

```

将有序数组转换为二叉搜索树

```

1  class Solution {
2  public:
3      TreeNode* sortedArrayToBST(vector<int>& nums) {
4          return helper(nums, 0, nums.size() - 1);
5      }
6
7      TreeNode* helper(vector<int>& nums, int left, int right) {
8          if (left > right) {
9              return nullptr;
10         }
11
12         // 总是选择中间位置左边的数字作为根节点
13         int mid = (left + right) / 2;
14
15         TreeNode* root = new TreeNode(nums[mid]);
16         root->left = helper(nums, left, mid - 1);
17         root->right = helper(nums, mid + 1, right);
18         return root;
19     }
20 };

```

验证二叉搜索树

```

1  class Solution {
2  public:
3      vector<int> vec;
4      void traversal(TreeNode* node){
5          if(node == NULL) return ;
6          traversal(node->left);
7          vec.push_back(node->val);
8          traversal(node->right);
9      }
10
11      bool isValidBST(TreeNode* root) {
12          if(root == NULL) return true;
13          traversal(root);
14          for(int i=1; i<vec.size(); ++i){
15              if(vec[i]<=vec[i-1]) return false;
16          }
17          return true;
18      }
19 };

```

二叉搜索树中第K小的元素

```

1  class Solution {

```

```

2 public:
3     int kthSmallest(TreeNode* root, int k) {
4         stack<TreeNode*> stack;
5         while (root != nullptr || stack.size() > 0) {
6             while (root != nullptr) {
7                 stack.push(root);
8                 root = root->left;
9             }
10            root = stack.top();
11            stack.pop();
12            --k;
13            if (k == 0) {
14                break;
15            }
16            root = root->right;
17        }
18        return root->val;
19    }
20 };

```

二叉搜索树的右视图

```

1 class Solution {
2 public:
3     vector<int> rightSideView(TreeNode* root) {
4         vector<int> result;
5         queue<TreeNode*> que;
6         if(root == NULL) return {};
7         que.push(root);
8         while(!que.empty()){
9             int size = que.size();
10            for(int i=0; i<size; ++i){
11                TreeNode* node = que.front();
12                que.pop();
13                if(node->left) que.push(node->left);
14                if(node->right) que.push(node->right);
15                if(i == size-1) result.push_back(node->val);
16            }
17        }
18        return result;
19    }
20 };

```

二叉树展开为链表

```

1 class Solution {
2 public:
3     vector<TreeNode*> vec;
4
5     void traversal(TreeNode* node){
6         if(node == NULL) return;
7
8         vec.push_back(node);
9         traversal(node->left);

```

```

10     traversal(node->right);
11 }
12
13 void flatten(TreeNode* root) {
14     traversal(root);
15     for(int i=1; i<vec.size(); ++i){
16         vec[i-1]->left = NULL;
17         vec[i-1]->right = vec[i];
18     }
19 }
20 };

```

从前序与中序遍历序列构造二叉树

本代码使用的是闭区间的理解方式，当 $pre_l < pre_r$ 的时候说明此时区间内只有单个元素，递归终止。

```

1  class Solution {
2  public:
3      unordered_map<int,int> index;
4      TreeNode* traversal(vector<int>& preorder, vector<int>& inorder, int
pre_l, int pre_r, int in_l, int in_r){
5          if(pre_r-pre_l < 0) return NULL; //仅剩单个元素的时候终止
6
7          int pre_node_index = pre_l;
8          int in_node_index = index[preorder[pre_l]];
9
10         TreeNode* root = new TreeNode(preorder[pre_l]);
11
12         int l_length = in_node_index - in_l;
13         int r_length = in_r - in_node_index;
14
15         root->left =
traversal(preorder, inorder, pre_l+1, pre_l+l_length, in_l, in_l+l_length-1);
16         root->right =
traversal(preorder, inorder, pre_l+l_length+1, pre_r, in_node_index+1, in_node_in
dex+r_length);
17
18         return root;
19     }
20 }
21
22     TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
23         int n = inorder.size();
24         for(int i=0; i<n; ++i){
25             index[inorder[i]] = i;
26         }
27         return traversal(preorder, inorder, 0, n-1, 0, n-1);
28     }
29 };

```

路径之和III

使用回溯加前缀和的做法

```

1  class Solution {
2  public:
3      unordered_map <long long,int> pre;
4
5      int backtracking(TreeNode* root,int targetSum,long long sum){
6          if(root == NULL) return 0;
7
8          int ret = 0;
9          sum += root->val;
10         if(pre.find(sum - targetSum) != pre.end()){
11             ret = pre[sum - targetSum];
12         }
13         pre[sum]++;
14
15         ret += backtracking(root->left,targetSum,sum);
16         ret += backtracking(root->right,targetSum,sum);
17
18         pre[sum]--;
19         sum -= root->val;
20
21         return ret;
22     }
23
24     int pathSum(TreeNode* root, int targetSum) {
25         pre[0] = 1;
26         return backtracking(root,targetSum,0);
27     }
28 };

```

二叉树最近公共祖先

自底向上的查询，使用后序遍历（天然的回溯过程），注意想象一下各类情况。

```

1  class Solution {
2  public:
3      TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q)
4      {
5          if(root == p || root == q || root == NULL) return root;
6
7          TreeNode* left = lowestCommonAncestor(root->left,p,q);
8          TreeNode* right = lowestCommonAncestor(root->right,p,q);
9          if(left && right) return root;
10         else if(!left && right) return right;
11         else if(left && !right) return left;
12         else return NULL;
13     }
14 };

```

二叉树中的最大路径和

注意这道题递归的返回值！

```

1  class Solution {
2  public:

```

```

3     int max_sum = INT_MIN;
4
5     int maxsum(TreeNode* root){
6         if(root == NULL) return 0;
7
8         int left_gain = max(maxsum(root->left),0);
9         int right_gain = max(maxsum(root->right),0);
10        int cur_sum = root->val + left_gain + right_gain;
11
12        max_sum = max(max_sum,cur_sum);
13        return root->val + max(left_gain,right_gain); //注意这个返回值
14    }
15
16    int maxPathSum(TreeNode* root) {
17        maxsum(root);
18        return max_sum;
19    }
20 };

```

回溯

全排列

这里使用了used数字以此区分数值是否被使用过。

```

1  class Solution {
2  public:
3      vector<vector<int>> result;
4      vector<int> path;
5      void backtracking(vector<int>& nums,vector<bool>& used){
6          if(path.size() == nums.size()){
7              result.push_back(path);
8              return ;
9          }
10
11         for(int i=0; i<nums.size(); ++i){
12             if(used[i] == true) continue;
13             path.push_back(nums[i]);
14             used[i] = true;
15             backtracking(nums,used);
16             used[i] = false;
17             path.pop_back();
18         }
19
20     }
21
22     vector<vector<int>> permute(vector<int>& nums) {
23         vector<bool> used(nums.size(),false);
24         backtracking(nums,used);
25         return result;
26     }
27 };

```

子集

注意收集子集的位置

```

1  class Solution {
2  private:
3      vector<vector<int>> result;
4      vector<int> path;
5      void backtracking(vector<int>& nums, int startIndex) {
6          result.push_back(path); // 收集子集，要放在终止添加的上面，否则会漏掉自己
7          if (startIndex >= nums.size()) { // 终止条件可以不加
8              return;
9          }
10         for (int i = startIndex; i < nums.size(); i++) {
11             path.push_back(nums[i]);
12             backtracking(nums, i + 1);
13             path.pop_back();
14         }
15     }
16 public:
17     vector<vector<int>> subsets(vector<int>& nums) {
18         result.clear();
19         path.clear();
20         backtracking(nums, 0);
21         return result;
22     }
23 };

```

电话号码的组合

```

1  class Solution {
2  public:
3      const string smap[8]={
4          "abc",
5          "def",
6          "ghi",
7          "jkl",
8          "mno",
9          "pqrs",
10         "tuv",
11         "wxyz",
12     };
13
14     vector<string> result;
15     string path;
16
17     void backtracking(string digits,int num){
18         if(path.size() == digits.size()){
19             result.push_back(path);
20             return ;
21         }
22         for(int i=0;i<smap[digits[num] - '2'].size();++i){
23             path += smap[digits[num] - '2'][i];
24             backtracking(digits,num+1);

```

```

25         path.pop_back();
26     }
27 }
28
29
30 vector<string> letterCombinations(string digits) {
31     if(digits.empty()) return result;
32     backtracking(digits,0);
33     return result;
34 }
35 };

```

组合总和

为什么这道题不能当作完全背包来做？因为需要把满足的排列都列出来！

```

1  class Solution {
2  public:
3      vector<vector<int>> result;
4      vector<int> path;
5      void backtracking(vector<int>& candidates, int target, int sum, int index)
6      {
7          if(sum > target) return ;
8          if(sum == target) result.push_back(path);
9
10         for(int i=index; i<candidates.size();++i){
11             sum += candidates[i];
12             path.push_back(candidates[i]);
13             backtracking(candidates, target, sum, i);
14             sum -= candidates[i];
15             path.pop_back();
16         }
17
18         vector<vector<int>> combinationSum(vector<int>& candidates, int target)
19         {
20             backtracking(candidates, target, 0, 0);
21             return result;
22         }
23     };

```

括号生成

```

1  class Solution {
2  public:
3      void backtrack(vector<string>& ans, string& cur, int open, int close,
4      int n) {
5          if (cur.size() == n * 2) {
6              ans.push_back(cur);
7              return;
8          }
9          if (open < n) {
10             cur.push_back('(');
11             backtrack(ans, cur, open + 1, close, n);

```

```

11         cur.pop_back();
12     }
13     if (close < open) {
14         cur.push_back('(');
15         backtrack(ans, cur, open, close + 1, n);
16         cur.pop_back();
17     }
18 }
19
20 vector<string> generateParenthesis(int n) {
21     vector<string> result;
22     string current;
23     backtrack(result, current, 0, 0, n);
24     return result;
25 }
26 };
27

```

单词搜索

```

1  class Solution {
2  public:
3      bool check(vector<vector<char>>& board, vector<vector<int>>& visited,
4      int i, int j, string& s, int k) {
5          if (board[i][j] != s[k]) {
6              return false;
7          } else if (k == s.length() - 1) {
8              return true;
9          }
10         visited[i][j] = true;
11         vector<pair<int, int>> directions{{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
12         bool result = false;
13         for (const auto& dir: directions) {
14             int newi = i + dir.first, newj = j + dir.second;
15             if (newi >= 0 && newi < board.size() && newj >= 0 && newj <
16             board[0].size()) {
17                 if (!visited[newi][newj]) {
18                     bool flag = check(board, visited, newi, newj, s, k + 1);
19                     if (flag) {
20                         result = true;
21                         break;
22                     }
23                 }
24             }
25             visited[i][j] = false;
26             return result;
27         }
28
29         bool exist(vector<vector<char>>& board, string word) {
30             int h = board.size(), w = board[0].size();
31             vector<vector<int>> visited(h, vector<int>(w));
32             for (int i = 0; i < h; i++) {
33                 for (int j = 0; j < w; j++) {
34                     bool flag = check(board, visited, i, j, word, 0);
35                 }
36             }
37             return result;
38         }
39     };
40 }

```

```

34         if (flag) {
35             return true;
36         }
37     }
38 }
39 return false;
40 }
41 };

```

分割回文串

```

1  class Solution {
2  public:
3      vector<vector<string>> result;
4      vector<string> path;
5
6      void backtracking(string s,int index){
7          if(index == s.size()){
8              result.push_back(path);
9              return ;
10         }
11         for(int i = index; i<s.size();++i){
12             if(ispalindrome(s,index,i)){
13                 string str = s.substr(index,i-index+1);
14                 path.push_back(str);
15             }
16             else continue;
17             backtracking(s,i+1);
18             path.pop_back();
19         }
20     }
21     bool ispalindrome(const string& s, int start, int end) {
22         for (int i = start, j = end; i < j; i++, j--) {
23             if (s[i] != s[j]) {
24                 return false;
25             }
26         }
27         return true;
28     }
29     vector<vector<string>> partition(string s) {
30         result.clear();
31         path.clear();
32         backtracking(s, 0);
33         return result;
34     }
35 };

```

二分查找

```

1  class Solution {
2  public:
3      int search(vector<int>& nums, int target) {
4          int left = 0;

```

```

5         int right = nums.size() - 1; // 定义target在左闭右闭的区间里, [left,
right]
6         while (left <= right) { // 当left==right, 区间[left, right]依然有效, 所
以用 <=
7             int middle = left + ((right - left) / 2); // 防止溢出 等同于(left +
right)/2
8             if (nums[middle] > target) {
9                 right = middle - 1; // target 在左区间, 所以[left, middle - 1]
10            } else if (nums[middle] < target) {
11                left = middle + 1; // target 在右区间, 所以[middle + 1, right]
12            } else { // nums[middle] == target
13                return middle; // 数组中找到目标值, 直接返回下标
14            }
15        }
16        // 未找到目标值
17        return -1;
18    }
19 };

```

搜索插入位置

```

1 class Solution {
2 public:
3     int searchInsert(vector<int>& nums, int target) {
4         auto iter = lower_bound(nums.begin(), nums.end(), target);
5         return iter - nums.begin();
6     }
7 };

```

搜索二维矩阵

```

1 class Solution {
2 public:
3     // 根据该矩阵的特性, 将整个二维数组看成一个单调递增的一维数组进行操作
4     bool searchMatrix(vector<vector<int>>& matrix, int target) {
5         // 记录 行m 和 列n
6         int m = matrix.size(), n = matrix[0].size();
7         // 二分的范围就是整个矩阵的元素个数[闭区间]
8         int left = 0, right = m * n - 1;
9         while (left <= right) {
10             int mid = left + (right - left) / 2;
11             // 记录mid所在的 行号i, 列号j
12             int i = mid / n, j = mid % n;
13             // left左边都是小于target的元素
14             if (matrix[i][j] < target)
15                 left = mid + 1;
16             else
17                 right = mid - 1;
18         }
19         // 返回时要考虑left是否在合法范围内
20         return left >= m * n ? false : matrix[left / n][left % n] == target;
21     }
22 };

```

在排序数组中查找元素第一个和最后一个位置

```

1  class Solution {
2  public:
3      vector<int> searchRange(vector<int>& nums, int target) {
4          // 查找起始位置 (第一个 >= target 的位置)
5          int left = lower_bound(nums.begin(), nums.end(), target) -
nums.begin();
6
7          // 如果起始位置越界或该位置的值不是 target, 说明 target 不存在
8          if (left == nums.size() || nums[left] != target) {
9              return {-1, -1};
10         }
11
12         // 查找结束位置 (第一个 > target 的位置 - 1)
13         int right = upper_bound(nums.begin(), nums.end(), target) -
nums.begin() - 1;
14
15         return {left, right};
16     }
17 };

```

搜索旋转排序数组

```

1  class Solution {
2  public:
3      int search(vector<int>& nums, int target) {
4          int n = nums.size();
5          int l = 0, r = n - 1;
6          while (l <= r) {
7              int mid = (l + r) / 2;
8              if (nums[mid] == target) return mid;
9              if (nums[0] <= nums[mid]) {
10                 if (nums[0] <= target && target < nums[mid]) {
11                     r = mid - 1;
12                 } else {
13                     l = mid + 1;
14                 }
15             } else {
16                 if (nums[mid] < target && target <= nums[n - 1]) {
17                     l = mid + 1;
18                 } else {
19                     r = mid - 1;
20                 }
21             }
22         }
23         return -1;
24     }
25 };

```

寻找旋转排序数组最小值

```

1  class Solution {
2  public:
3      int findMin(vector<int>& nums) {
4          int low = 0;
5          int high = nums.size() - 1;
6          while (low < high) {
7              int pivot = (high + low) / 2;
8              if (nums[pivot] < nums[high]) {
9                  high = pivot;
10             }
11             else {
12                 low = pivot + 1;
13             }
14         }
15         return nums[low];
16     }
17 };

```

栈

有效的括号

```

1  class Solution {
2  public:
3      bool isValid(string s) {
4          if (s.size() % 2 != 0) return false; // 如果s的长度为奇数，一定不符合要求
5          stack<char> st;
6          for (int i = 0; i < s.size(); i++) {
7              if (s[i] == '(') st.push('(');
8              else if (s[i] == '{') st.push('{');
9              else if (s[i] == '[') st.push('[');
10             // 第三种情况：遍历字符串匹配的过程中，栈已经为空了，没有匹配的字符了，说明右
             括号没有找到对应的左括号 return false
11             // 第二种情况：遍历字符串匹配的过程中，发现栈里没有我们要匹配的字符。所以
             return false
12             else if (st.empty() || st.top() != s[i]) return false;
13             else st.pop(); // st.top() 与 s[i]相等，栈弹出元素
14         }
15         // 第一种情况：此时我们已经遍历完了字符串，但是栈不为空，说明有相应的左括号没有右
             括号来匹配，所以return false，否则就return true
16         return st.empty();
17     }
18 };

```

最小栈

```

1  class MinStack {
2      stack<int> x_stack;
3      stack<int> min_stack;
4  public:

```

```

5     Minstack() {
6         min_stack.push(INT_MAX);
7     }
8
9     void push(int x) {
10        x_stack.push(x);
11        min_stack.push(min(min_stack.top(), x));
12    }
13
14    void pop() {
15        x_stack.pop();
16        min_stack.pop();
17    }
18
19    int top() {
20        return x_stack.top();
21    }
22
23    int getMin() {
24        return min_stack.top();
25    }
26 };

```

字符串解码

```

1  class Solution {
2  public:
3      string getDigits(string &s, size_t &ptr) {
4          string ret = "";
5          while (isdigit(s[ptr])) {
6              ret.push_back(s[ptr++]);
7          }
8          return ret;
9      }
10
11     string getString(vector <string> &v) {
12         string ret;
13         for (const auto &s: v) {
14             ret += s;
15         }
16         return ret;
17     }
18
19     string decodeString(string s) {
20         vector <string> stk;
21         size_t ptr = 0;
22
23         while (ptr < s.size()) {
24             char cur = s[ptr];
25             if (isdigit(cur)) {
26                 // 获取一个数字并进栈
27                 string digits = getDigits(s, ptr);
28                 stk.push_back(digits);
29             } else if (isalpha(cur) || cur == '[') {
30                 // 获取一个字母并进栈

```

```

31         stk.push_back(string(1, s[ptr++]));
32     } else {
33         ++ptr;
34         vector<string> sub;
35         while (stk.back() != "[") {
36             sub.push_back(stk.back());
37             stk.pop_back();
38         }
39         reverse(sub.begin(), sub.end());
40         // 左括号出栈
41         stk.pop_back();
42         // 此时栈顶为当前 sub 对应的字符串应该出现的次数
43         int repTime = stoi(stk.back());
44         stk.pop_back();
45         string t, o = getString(sub);
46         // 构造字符串
47         while (repTime--) t += o;
48         // 将构造好的字符串入栈
49         stk.push_back(t);
50     }
51 }
52
53 return getString(stk);
54 }
55 };

```

每日温度

```

1  class Solution {
2  public:
3      vector<int> dailyTemperatures(vector<int>& temperatures) {
4          stack<int> st;
5          vector<int> result(temperatures.size(), 0);
6          st.push(0);
7          for(int i=1; i<temperatures.size(); ++i){
8              if(temperatures[i]<temperatures[st.top()]){
9                  st.push(i);
10             }
11             else if(temperatures[i]==temperatures[st.top()]){
12                 st.push(i);
13             }
14             else{
15                 while(!st.empty() && temperatures[i]>temperatures[st.top()])
16                 {
17                     result[st.top()] = i - st.top();
18                     st.pop();
19                 }
20                 st.push(i);
21             }
22         }
23         return result;
24     };

```

柱状图中最大的矩形

```

1  class Solution {
2  public:
3      int largestRectangleArea(vector<int>& heights) {
4          int result = 0;
5          stack<int> st;
6          heights.insert(heights.begin(), 0); // 数组头部加入元素0
7          heights.push_back(0); // 数组尾部加入元素0
8          st.push(0);
9
10         // 第一个元素已经入栈，从下标1开始
11         for (int i = 1; i < heights.size(); i++) {
12             if (heights[i] > heights[st.top()]) { // 情况一
13                 st.push(i);
14             } else if (heights[i] == heights[st.top()]) { // 情况二
15                 st.pop(); // 这个可以加，可以不加，效果一样，思路不同
16                 st.push(i);
17             } else { // 情况三
18                 while (!st.empty() && heights[i] < heights[st.top()]) { // 注
19                     // 意是while
20                     int mid = st.top();
21                     st.pop();
22                     if (!st.empty()) {
23                         int left = st.top();
24                         int right = i;
25                         int w = right - left - 1;
26                         int h = heights[mid];
27                         result = max(result, w * h);
28                     }
29                     st.push(i);
30                 }
31             }
32             return result;
33         }
34     };

```

堆

数组中第k个最大元素

```

1  class Solution {
2  public:
3      void maxHeapify(vector<int>& a, int i, int heapSize) {
4          int l = i * 2 + 1, r = i * 2 + 2, largest = i;
5          if (l < heapSize && a[l] > a[largest]) {
6              largest = l;
7          }
8          if (r < heapSize && a[r] > a[largest]) {
9              largest = r;
10         }
11         if (largest != i) {

```

```

12         swap(a[i], a[largest]);
13         maxHeapify(a, largest, heapSize);
14     }
15 }
16
17 void buildMaxHeap(vector<int>& a, int heapSize) {
18     for (int i = heapSize / 2 - 1; i >= 0; --i) {
19         maxHeapify(a, i, heapSize);
20     }
21 }
22
23 int findKthLargest(vector<int>& nums, int k) {
24     int heapSize = nums.size();
25     buildMaxHeap(nums, heapSize);
26     for (int i = nums.size() - 1; i >= nums.size() - k + 1; --i) {
27         swap(nums[0], nums[i]);
28         --heapSize;
29         maxHeapify(nums, 0, heapSize);
30     }
31     return nums[0];
32 }
33 };

```

前K个高频元素

```

1  class Solution {
2  public:
3      void maxHeapify(vector<int>& a, int i, int heapSize) {
4          int l = i * 2 + 1, r = i * 2 + 2, largest = i;
5          if (l < heapSize && a[l] > a[largest]) {
6              largest = l;
7          }
8          if (r < heapSize && a[r] > a[largest]) {
9              largest = r;
10         }
11         if (largest != i) {
12             swap(a[i], a[largest]);
13             maxHeapify(a, largest, heapSize);
14         }
15     }
16
17     void buildMaxHeap(vector<int>& a, int heapSize) {
18         for (int i = heapSize / 2 - 1; i >= 0; --i) {
19             maxHeapify(a, i, heapSize);
20         }
21     }
22
23     int findKthLargest(vector<int>& nums, int k) {
24         int heapSize = nums.size();
25         buildMaxHeap(nums, heapSize);
26         for (int i = nums.size() - 1; i >= nums.size() - k + 1; --i) {
27             swap(nums[0], nums[i]);
28             --heapSize;
29             maxHeapify(nums, 0, heapSize);
30         }

```

```

31         return nums[0];
32     }
33 };

```

贪心算法

买股票最佳时机

```

1  //贪心解法
2  class Solution {
3  public:
4      int maxProfit(vector<int>& prices) {
5          int low = INT_MAX;
6          int result = 0;
7          for (int i = 0; i < prices.size(); i++) {
8              low = min(low, prices[i]); // 取最左最小价格
9              result = max(result, prices[i] - low); // 直接取最大区间利润
10         }
11         return result;
12     }
13 };

```

跳跃游戏

```

1  class Solution {
2  public:
3      bool canJump(vector<int>& nums) {
4          int cover = 0;
5          if (nums.size() == 1) return true; // 只有一个元素，就是能达到
6          for (int i = 0; i <= cover; i++) { // 注意这里是小于等于cover
7              cover = max(i + nums[i], cover);
8              if (cover >= nums.size() - 1) return true; // 说明可以覆盖到终点了
9          }
10         return false;
11     }
12 };

```

跳跃游戏II

```

1  class Solution {
2  public:
3      int jump(vector<int>& nums) {
4          int maxPos = 0, n = nums.size(), end = 0, step = 0;
5          for (int i = 0; i < n - 1; ++i) { // 注意这里只控制到移动到终点前一步
6              if (maxPos >= i) {
7                  maxPos = max(maxPos, i + nums[i]);
8                  if (i == end) {
9                      end = maxPos;
10                     ++step;
11                 }
12             }
13         }
14     }
15 };

```

```

12         }
13     }
14     return step;
15 }
16 };

```

划分字母区间

```

1  class Solution {
2  public:
3      vector<int> partitionLabels(string s) {
4          int last[26];
5          int length = s.size();
6          for (int i = 0; i < length; i++) {
7              last[s[i] - 'a'] = i;
8          }
9          vector<int> partition;
10         int start = 0, end = 0;
11         for (int i = 0; i < length; i++) {
12             end = max(end, last[s[i] - 'a']);
13             if (i == end) {
14                 partition.push_back(end - start + 1);
15                 start = end + 1;
16             }
17         }
18         return partition;
19     }
20 };

```

动态规划

基础知识

动态规划五部曲

1. 确定dp数组（dp table）以及下标的含义，常见的包括 **1.求组合数 2.求排列数 3.求最小数**
2. 确定递推公式，根据题目关系来
3. dp数组如何初始化，要根据递推公式，观察哪些值需要提前给好
4. 确定遍历顺序，**如果求组合数就是外层for循环遍历物品，内层for遍历背包。如果求排列数就是外层for遍历背包，内层for循环遍历物品。**
5. 举例推导dp数组

0-1背包问题

有n件物品和一个最多能背重量为w 的背包。第i件物品的重量是weight[i]，得到的价值是value[i]。**每件物品只能用一次**，求解将哪些物品装入背包里物品价值总和最大。需要使用二维的动态规划(物品为行，容量为列)进行求解：

五部曲如下：

- 1.dp[i][j] 表示从下标为[0-i]的物品里任意取，放进容量为j的背包，价值总和最大是多少。

- 2.递推公式: $dp[i][j] = \max(dp[i-1][j], dp[i-1][j - \text{weight}[i]] + \text{value}[i])$;表示考虑前*i*个物品情况下, 放或者不放当前物品。需要获得左上的元素,注意 $j - \text{weight}[i]$ 可能小于0, 针对这种情况, 直接另 $dp[i][j] = dp[i-1][j]$
- 3.初始化问题: 当背包容量为0的时候, 价值总和只能为0, 当只考虑一个物品的时候, $dp[0][j], j < \text{weight}$ 的时候为0, 大于后为 $\text{value}[0]$
- 4.确定迭代顺序, 由于需要左上元素, 因此从上到下, 从左到右
- 5.检查数组正确性

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, bagweight; // bagweight代表行李箱空间
6
7      cin >> n >> bagweight;
8
9      vector<int> weight(n, 0); // 存储每件物品所占空间
10     vector<int> value(n, 0); // 存储每件物品价值
11
12     for(int i = 0; i < n; ++i) {
13         cin >> weight[i];
14     }
15     for(int j = 0; j < n; ++j) {
16         cin >> value[j];
17     }
18     // dp数组, dp[i][j]代表行李箱空间为j的情况下,从下标为[0, i]的物品里面任意取,能达到
    的最大价值
19     vector<vector<int>> dp(weight.size(), vector<int>(bagweight + 1, 0));
20
21     // 初始化, 因为需要用到dp[i - 1]的值
22     // j < weight[0]已在上方被初始化为0
23     // j >= weight[0]的值就初始化为value[0]
24     for (int j = weight[0]; j <= bagweight; j++) {
25         dp[0][j] = value[0];
26     }
27     for(int i = 1; i < weight.size(); i++) { // 遍历科研物品
28         for(int j = 0; j <= bagweight; j++) { // 遍历行李箱容量
29             if (j < weight[i]) dp[i][j] = dp[i - 1][j]; // 如果装不下这个物品,那
    么就继承dp[i - 1][j]的值
30             else {
31                 dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] +
    value[i]);
32             }
33         }
34     }
35     cout << dp[n - 1][bagweight] << endl;
36
37     return 0;
38 }

```

可以利用滚动数组类似的方法, 将dp简化为一维的 (注意由此改变的遍历顺序), 这里只简单给出代码

```

1  for (int i = 0; i < M; ++i) {
2      // 内层循环从 N 空间逐渐减少到当前研究材料所占空间
3      for (int j = N; j >= costs[i]; --j) {
4          // 考虑当前研究材料选择不选择的情况，选择最大值
5          dp[j] = max(dp[j], dp[j - costs[i]] + values[i]);
6      }
7  }

```

->分割等和子集：背包问题，不仅可以求 背包能被的最大价值，**可以求这个背包是否可以装满。**

完全背包问题

有N件物品和一个最多能背重量为W的背包。第i件物品的重量是weight[i]，得到的价值是value[i]。**每件物品都有无限个（也就是可以放入背包多次），求解将哪些物品装入背包里物品价值总和最大。完全背包和01背包问题唯一不同的地方就是，每种物品有无限件。**

五部曲如下：

1.dp[i][j] 表示从下标为[0-i]的物品，每个物品可以取无限次，放进容量为j的背包，价值总和最大是多少。

2.dp[i][j] = max(dp[i - 1][j], dp[i][j - weight[i]] + value[i]);还是针对两种情况：放or不放当前物品，需要左边以及上方元素，背包不够时直接用上方元素

3.初始化：第一行数据初始化，对背包大小为0时进行初始化

4.遍历顺序，从上到下，从左到右

5.检查数组正确性

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      int n, bagweight;
7      int w, v;
8      cin >> n >> bagweight;
9      vector<int> weight(n);
10     vector<int> value(n);
11     for (int i = 0; i < n; i++) {
12         cin >> weight[i] >> value[i];
13     }
14
15     vector<vector<int>> dp(n, vector<int>(bagweight + 1, 0));
16
17     // 初始化
18     for (int j = weight[0]; j <= bagweight; j++)
19         dp[0][j] = dp[0][j - weight[0]] + value[0];
20
21     for (int i = 1; i < n; i++) { // 遍历物品
22         for(int j = 0; j <= bagweight; j++) { // 遍历背包容量
23             if (j < weight[i]) dp[i][j] = dp[i - 1][j];
24             else dp[i][j] = max(dp[i - 1][j], dp[i][j - weight[i]] +
value[i]);
25         }
26     }
27 }

```

```

26     }
27
28     cout << dp[n - 1][bagweight] << endl;
29
30     return 0;
31 }

```

爬楼梯（斐波那契数列）

- 1.dp[i] 代表爬到第i级台阶的总方法数
- 2.dp[i]的状态有前一步状态决定（前一级或前两级） $dp[i]=dp[i-1]+dp[i-2]$,所以可以只维护前两个值
- 3.初始化 $dp[0]=1,dp[1]=1$
- 4.遍历顺序，i从2开始逐步变大直到第n级台阶
- 5.举例推导

```

1  class Solution {
2  public:
3      int climbStairs(int n) {
4          if (n <= 1) return n; // 因为下面直接对dp[2]操作了，防止空指针
5          vector<int> dp(n + 1);
6          dp[0] = 1;
7          dp[1] = 1;
8          for (int i = 2; i <= n; i++) { // 注意i是从3开始的
9              dp[i] = dp[i - 1] + dp[i - 2];
10         }
11         return dp[n];
12     }
13 };

```

杨辉三角

- 1.dp[i][j]代表杨辉三角中第i行第j个元素的值，从0开始
- 2.递推公式： $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$ ，需要具有左上角的值，所以从上到下，从左到右
- 3.初始化所有值为1
- 4.从上到下，从左到右进行遍历
- 5.举例推导

```

1  class Solution {
2  public:
3      vector<vector<int>> generate(int numRows) {
4          vector<vector<int>> ret(numRows);
5          for (int i = 0; i < numRows; ++i) {
6              ret[i].resize(i + 1);
7              ret[i][0] = ret[i][i] = 1;
8              for (int j = 1; j < i; ++j) {
9                  ret[i][j] = ret[i - 1][j] + ret[i - 1][j - 1];
10             }
11         }
12         return ret;
13     }
14 };

```

```

13     }
14 };

```

打家劫舍

- 1.dp[i]代表直到前i个房屋内，能搜集到的最大金币
- 2.递归公式：如果偷了前一家就没法偷这一家了，因此 $dp[i] = \max(dp[i-2]+nums[i], dp[i-1])$ ，需要前两个元素
- 3.初始化，因为需要前两个元素，所以 $dp[0]=nums[0], dp[1]=\max(nums[0], nums[1])$
- 4.遍历顺序：根据递推关系，i应该从小到大进行遍历
- 5.检查数组

```

1  class Solution {
2  public:
3      int rob(vector<int>& nums) {
4          if (nums.size() == 0) return 0;
5          if (nums.size() == 1) return nums[0];
6          vector<int> dp(nums.size());
7          dp[0] = nums[0];
8          dp[1] = max(nums[0], nums[1]);
9          for (int i = 2; i < nums.size(); i++) {
10             dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);
11         }
12         return dp[nums.size() - 1];
13     }
14 };

```

完全平方数

乍一看没什么关系，实际可以抽象成完全背包问题：**将完全平方数看作是物品重量，如何用最少物品刚好凑满这个背包？**

- 1.确认dp[j]：最少的使用完全平方数凑成j的个数
- 2.递推关系：dp[i][j]表示使用前i个数字凑齐j的方法， $dp[i][j] = \min(dp[i][j - i*i], dp[i-1][j])$ ，需要上一行以及左边的元素
- 3.确认初始化方法，由于要求最小值，所以所有元素初始化为INT_MAX，对第一列元素赋值为0
- 4.确认遍历顺序，从上到下，从左到右进行遍历
- 5.检查数组是否正确

```

1  //1维dp写法
2  class Solution {
3  public:
4      int numSquares(int n) {
5          vector<int> dp(n + 1, INT_MAX);
6          dp[0] = 0;
7          for (int i = 1; i * i <= n; i++) { // 遍历物品
8              for (int j = i * i; j <= n; j++) { // 遍历背包
9                  dp[j] = min(dp[j - i * i] + 1, dp[j]);
10             }

```

```

11     }
12     return dp[n];
13 }
14 };
15
16 //2维dp写法
17 class Solution {
18 public:
19     int numSquares(int n) {
20         int m = sqrt(n);
21         vector<vector<int>> dp(m+1,vector<int> (n+1,INT_MAX));
22         for (int i=1; i < dp.size(); ++i){
23             dp[i][0] = 0;
24         }
25         for(int i=1; i< dp.size();i++){
26             for(int j=1; j<=n; ++j){
27                 if(j < i*i) dp[i][j] = dp[i-1][j];
28                 else dp[i][j] = min(dp[i][j-i*i]+1 , dp[i-1][j]);
29             }
30         }
31         return dp[m][n];
32     }
33 };

```

零钱兑换

方法和完全平方数几乎一致，在此不重复说明

```

1  class Solution {
2  public:
3      int coinChange(vector<int>& coins, int amount) {
4          vector<int> dp(amount+1,INT_MAX);
5          for(int i=0; i<=amount; ++i){
6              if(i%coins[0] == 0) dp[i] = i/coins[0];
7          }
8          for(int i=1; i<coins.size(); ++i){
9              for(int j=coins[i]; j<=amount; j++){
10                 if (dp[j - coins[i]] != INT_MAX){//关键条件，防止溢出
11                     dp[j] = min(dp[j],dp[j-coins[i]]+1);
12                 }
13             }
14         }
15         if(dp[amount] == INT_MAX) return -1;
16         else return dp[amount];
17     }
18 };

```

单词拆分

- 1.dp含义：dp[i] 字符串长度为i的话，dp[i]为true，表示可以拆分为一个或多个在字典中出现的单词。
- 2.dp 递推关系：if dp[j]为真 并且[j,i]区间上的字串找得到的话，那么dp[i] = True，既然要有一个区间的头为真，那么dp[0]要么真
- 3.dp初始化：dp[0]为True，其他都为false

4.确认遍历顺序：先对背包进行遍历！再对物品进行遍历

5.检查数组

```

1  class Solution {
2  public:
3      bool wordBreak(string s, vector<string>& wordDict) {
4          vector<bool> dp(s.size()+1, false);
5          dp[0] = true;
6          for(int i=1; i<=s.size(); i++){
7              for(auto str:wordDict){
8                  int strlen = str.length();
9                  if(i < strlen) continue;
10                 if(s.substr(i-strlen, strlen)==str && dp[i-strlen] == true){
11                     dp[i] = true;
12                 }
13             }
14         }
15         return dp[s.size()];
16     }
17 };

```

最长递增子序列

1.确认dp数组的含义，dp[j]代表到j位置能够满足的最长递增子序列

2.dp[j] = max(dp[i])+1 i<j并且nums[i]>nums[j]

3.dp[0] = 1, 并且所有的数字都初始化为1

4.从i=1开始，每次访问i前面的元素

```

1  class Solution {
2  public:
3      int lengthOfLIS(vector<int>& nums) {
4          vector<int> dp(nums.size(), 1);
5          for(int i=1; i<nums.size(); ++i){
6              for(int j=0; j<i; ++j){
7                  if(nums[j]<nums[i]){
8                      dp[i] = max(dp[i], dp[j]+1);
9                  }
10             }
11         }
12         return *max_element(dp.begin(), dp.end());
13     }
14 };

```

乘积最大子数组

1.dp[i]表示到i位为止(以i为结尾的)最大的连续子数组

2.dp[i] = dp[i-1]*nums[i], nums[i]

3.dp[0]=nums[0]

4.i从1开始遍历。。。

这对吗？

实则不然，因为考虑到有负数的情况，负负得正，你怎么知道负数就一定要丢掉？所以最小子问题并不是这样，实际上，我们还要维护一个最小乘积单防负数情况

```

1  class Solution {
2  public:
3      int maxProduct(vector<int>& nums) {
4          vector<long> maxF(nums.begin(), nums.end()), minF(nums.begin(),
5              nums.end());
6          for (int i = 1; i < nums.size(); ++i) {
7              maxF[i] = max(maxF[i - 1] * nums[i], max((long)nums[i], minF[i -
8                  1] * nums[i]));
9              minF[i] = min(minF[i - 1] * nums[i], min((long)nums[i], maxF[i -
10                  1] * nums[i]));
11          }
12          return *max_element(maxF.begin(), maxF.end());
13      }
14  };

```

分割等和数组

把这道题看成特殊的01背包问题，求总和的一半能否被装满----类似问题：最后一块石头的重量

```

1  class Solution {
2  public:
3      bool canPartition(vector<int>& nums) {
4          vector<int> dp(10001, 0);
5          int sum = accumulate(nums.begin(), nums.end(), 0);
6          if (sum % 2 == 1) return false;
7          int target = sum / 2;
8
9          // 开始 01背包，注意j的遍历是倒序
10         for(int i = 0; i < nums.size(); i++) {
11             for(int j = target; j >= nums[i]; j--) { // 每一个元素一定是不可重复
12                 // 放入，所以从大到小遍历
13                 dp[j] = max(dp[j], dp[j - nums[i]] + nums[i]);
14             }
15         }
16         if (dp[target] == target) return true;
17         return false;
18     }
19 };

```

最长有效括号

```

1  class Solution {
2  public:
3      int longestValidParentheses(string s) {
4          int maxans = 0, n = s.length();
5          vector<int> dp(n, 0);
6          for (int i = 1; i < n; i++) {
7              if (s[i] == ')') {
8                  if (s[i - 1] == '(') {

```

```

9         dp[i] = (i >= 2 ? dp[i - 2] : 0) + 2;
10    } else if (i - dp[i - 1] > 0 && s[i - dp[i - 1] - 1] == '(')
11    {
12        dp[i] = dp[i - 1] + ((i - dp[i - 1]) >= 2 ? dp[i - dp[i
- 1] - 2] : 0) + 2;
12    }
13    maxans = max(maxans, dp[i]);
14    }
15    }
16    return maxans;
17    }
18    };

```

多维动态规划

不同路径

```

1  class Solution {
2  public:
3      int uniquePaths(int m, int n) {
4          vector<vector<int>> dp(m,vector<int>(n,0));
5          for(int i=0; i<m; ++i) dp[i][0] = 1;
6          for(int j=0; j<n; ++j) dp[0][j] = 1;
7          for(int i=1; i<m;++i){
8              for(int j=1; j<n;++j){
9                  dp[i][j]=dp[i-1][j] + dp[i][j-1];
10             }
11         }
12         return dp[m-1][n-1];
13     }
14 };

```

最小路径和

```

1  class Solution {
2  public:
3      int minPathSum(vector<vector<int>>& grid) {
4          if (grid.size() == 0 || grid[0].size() == 0) {
5              return 0;
6          }
7          int rows = grid.size(), columns = grid[0].size();
8          auto dp = vector < vector <int> > (rows, vector <int> (columns));
9          dp[0][0] = grid[0][0];
10         for (int i = 1; i < rows; i++) {
11             dp[i][0] = dp[i - 1][0] + grid[i][0];
12         }
13         for (int j = 1; j < columns; j++) {
14             dp[0][j] = dp[0][j - 1] + grid[0][j];
15         }
16         for (int i = 1; i < rows; i++) {
17             for (int j = 1; j < columns; j++) {
18                 dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + grid[i][j];
19             }

```

```

20     }
21     return dp[rows - 1][columns - 1];
22 }
23 };

```

最长回文子串

- 1.dp[i][j]表示i到j内为回文字符串
- 2.dp[i][j] = dp[i+1][j-1] && s[i] == s[j] 当 i=j i+1=j j-i>=2情况下讨论，由递推公式可知，遍历次序应该从下到上，从左到右
- 3.一开始的值全部设置为false。想象一个右上三角矩阵，进行动态规划
- 4.根据递推关系，从下到上，从左到右进行遍历

```

1  class Solution {
2  public:
3      string longestPalindrome(string s) {
4          vector<vector<int>> dp(s.size(), vector<int>(s.size(), 0));
5          int maxlenh = 0;
6          int left = 0;
7          int right = 0;
8          for (int i = s.size() - 1; i >= 0; i--) {
9              for (int j = i; j < s.size(); j++) {
10                 if (s[i] == s[j]) {
11                     if (j - i <= 1) { // 情况一 和 情况二
12                         dp[i][j] = true;
13                     } else if (dp[i + 1][j - 1]) { // 情况三
14                         dp[i][j] = true;
15                     }
16                 }
17                 if (dp[i][j] && j - i + 1 > maxlenh) {
18                     maxlenh = j - i + 1;
19                     left = i;
20                     right = j;
21                 }
22             }
23         }
24         return s.substr(left, right - left + 1);
25     }
26 };

```

最长公共子序列

- 1.dp[i][j]表示在字符串1 i位置到字符串2 j位置的最大公共子序列长度
- 2.dp[i][j] = if (t1[i] == t2[j]) dp=dp[i-1][j-1] + 1 不等于的话，就是取dp[i-1][j] 和 dp[i][j-1]更大的那个
- 3.初始化都为0，并且尺寸大一个维度
- 4.从上到下，从左到右进行遍历

```

1  class Solution {
2  public:
3      int longestCommonSubsequence(string text1, string text2) {

```

```

4         vector<vector<int>> dp(text1.size() + 1, vector<int>(text2.size() +
1, 0));
5         for (int i = 1; i <= text1.size(); i++) {
6             for (int j = 1; j <= text2.size(); j++) {
7                 if (text1[i - 1] == text2[j - 1]) {
8                     dp[i][j] = dp[i - 1][j - 1] + 1;
9                 } else {
10                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
11                }
12            }
13        }
14        return dp[text1.size()][text2.size()];
15    }
16 };

```

编辑距离

```

1  class Solution {
2  public:
3      int minDistance(string word1, string word2) {
4          vector<vector<int>> dp(word1.size() + 1, vector<int>(word2.size() +
1, 0));
5          for (int i = 0; i <= word1.size(); i++) dp[i][0] = i;
6          for (int j = 0; j <= word2.size(); j++) dp[0][j] = j;
7          for (int i = 1; i <= word1.size(); i++) {
8              for (int j = 1; j <= word2.size(); j++) {
9                  if (word1[i - 1] == word2[j - 1]) {
10                     dp[i][j] = dp[i - 1][j - 1];
11                 }
12                 else {
13                     dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j
- 1]}) + 1;
14                 }
15             }
16         }
17         return dp[word1.size()][word2.size()];
18     }
19 };

```

图论（含扩展）

深度优先搜索代码模板

```

1 void dfs(参数) {
2     if (终止条件) {
3         存放结果;
4         return;
5     }
6
7     for (选择: 本节点所连接的其他节点) {
8         处理节点;
9         dfs(图, 选择的节点); // 递归
10        回溯, 撤销处理结果 //根据题目需求来: 是否需要回头??
11    }
12 }

```

广度优先搜索代码模板

在无权图中，用广搜求最短路最为合适，广搜只要搜到了终点，那么一定是最短的路径。

```

1 int dir[4][2] = {0, 1, 1, 0, -1, 0, 0, -1}; // 表示四个方向
2 // grid 是地图, 也就是一个二维数组
3 // visited标记访问过的节点, 不要重复访问
4 // x,y 表示开始搜索节点的下标
5 void bfs(vector<vector<char>>& grid, vector<vector<bool>>& visited, int x,
6 int y) {
7     queue<pair<int, int>> que; // 定义队列
8     que.push({x, y}); // 起始节点加入队列
9     visited[x][y] = true; // 只要加入队列, 立刻标记为访问过的节点
10    while(!que.empty()) { // 开始遍历队列里的元素
11        pair<int, int> cur = que.front(); que.pop(); // 从队列取元素
12        int curx = cur.first;
13        int cury = cur.second; // 当前节点坐标
14        for (int i = 0; i < 4; i++) { // 开始想当前节点的四个方向左右上下去遍历
15            int nextx = curx + dir[i][0];
16            int nexty = cury + dir[i][1]; // 获取周边四个方向的坐标
17            if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
18                grid[0].size()) continue; // 坐标越界了, 直接跳过
19            if (!visited[nextx][nexty]) { // 如果节点没被访问过
20                que.push({nextx, nexty}); // 队列添加该节点为下一轮要遍历的节点
21                visited[nextx][nexty] = true; // 只要加入队列立刻标记, 避免重复访问
22            }
23        }
24    }
25 }

```

岛屿数量问题：深度搜索版本

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int dir[4][2] = {0, 1, 1, 0, -1, 0, 0, -1}; // 四个方向
5  void dfs(const vector<vector<int>>& grid, vector<vector<bool>>& visited, int
x, int y) {
6      if (visited[x][y] || grid[x][y] == 0) return; // 终止条件: 访问过的节点 或者
遇到海水
7      visited[x][y] = true; // 标记访问过
8      for (int i = 0; i < 4; i++) {
9          int nextx = x + dir[i][0];
10         int nexty = y + dir[i][1];
11         if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
grid[0].size()) continue; // 越界了, 直接跳过
12         dfs(grid, visited, nextx, nexty);
13     }
14 }
15
16 int main() {
17     int n, m;
18     cin >> n >> m;
19     vector<vector<int>> grid(n, vector<int>(m, 0));
20     for (int i = 0; i < n; i++) {
21         for (int j = 0; j < m; j++) {
22             cin >> grid[i][j];
23         }
24     }
25
26     vector<vector<bool>> visited(n, vector<bool>(m, false));
27
28     int result = 0;
29     for (int i = 0; i < n; i++) {
30         for (int j = 0; j < m; j++) {
31             if (!visited[i][j] && grid[i][j] == 1) {
32                 result++; // 遇到没访问过的陆地, +1
33                 dfs(grid, visited, i, j); // 将与其链接的陆地都标记上 true
34             }
35         }
36     }
37     cout << result << endl;
38 }

```

岛屿数量问题：广度搜索版本

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  int dir[4][2] = {0, 1, 1, 0, -1, 0, 0, -1}; // 四个方向
7  void bfs(const vector<vector<int>>& grid, vector<vector<bool>>& visited, int
x, int y) {
8      queue<pair<int, int>> que;
9      que.push({x, y});
10     visited[x][y] = true; // 只要加入队列, 立刻标记

```

```

11     while(!que.empty()) {
12         pair<int ,int> cur = que.front(); que.pop();
13         int curx = cur.first;
14         int cury = cur.second;
15         for (int i = 0; i < 4; i++) {
16             int nextx = curx + dir[i][0];
17             int nexty = cury + dir[i][1];
18             if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
grid[0].size()) continue; // 越界了, 直接跳过
19             if (!visited[nextx][nexty] && grid[nextx][nexty] == 1) {
20                 que.push({nextx, nexty});
21                 visited[nextx][nexty] = true; // 只要加入队列立刻标记
22             }
23         }
24     }
25 }
26
27 int main() {
28     int n, m;
29     cin >> n >> m;
30     vector<vector<int>> grid(n, vector<int>(m, 0));
31     for (int i = 0; i < n; i++) {
32         for (int j = 0; j < m; j++) {
33             cin >> grid[i][j];
34         }
35     }
36
37     vector<vector<bool>> visited(n, vector<bool>(m, false));
38
39     int result = 0;
40     for (int i = 0; i < n; i++) {
41         for (int j = 0; j < m; j++) {
42             if (!visited[i][j] && grid[i][j] == 1) {
43                 result++; // 遇到没访问过的陆地, +1
44                 bfs(grid, visited, i, j); // 将与其链接的陆地都标记上 true
45             }
46         }
47     }
48
49     cout << result << endl;
50 }
51 }

```

腐烂的橘子 (BFS)

```

1  class Solution {
2  public:
3      int orangesRotting(vector<vector<int>>& grid) {
4          int min = 0, fresh = 0;
5          queue<pair<int, int>> q;
6          for(int i = 0; i < grid.size(); i++) {
7              for(int j = 0; j < grid[0].size(); j++)
8                  if(grid[i][j] == 1) fresh++;
9                  else if(grid[i][j] == 2) q.push({i, j});
10         }

```

```

11     vector<pair<int, int>> dirs = { {-1, 0}, {1, 0}, {0, -1}, {0, 1} };
12     while(!q.empty()) {
13         int n = q.size();
14         bool rotten = false;
15         for(int i = 0; i < n; i++) {
16             auto x = q.front();
17             q.pop();
18             for(auto cur: dirs) {
19                 int i = x.first + cur.first;
20                 int j = x.second + cur.second;
21                 if(i >= 0 && i < grid.size() && j >= 0 && j <
grid[0].size() && grid[i][j] == 1) {
22                     grid[i][j] = 2;
23                     q.push({i, j});
24                     fresh--;
25                     rotten = true;
26                 }
27             }
28         }
29         if(rotten) min++;
30     }
31     return fresh ? -1 : min;
32 }
33 };

```

课程表 (拓扑排序)

```

1  class Solution {
2  private:
3      vector<vector<int>> edges;
4      vector<int> indeg;
5
6  public:
7      bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
8          edges.resize(numCourses);
9          indeg.resize(numCourses);
10         for (const auto& info: prerequisites) {
11             edges[info[1]].push_back(info[0]);
12             ++indeg[info[0]];
13         }
14
15         queue<int> q;
16         for (int i = 0; i < numCourses; ++i) {
17             if (indeg[i] == 0) {
18                 q.push(i);
19             }
20         }
21
22         int visited = 0;
23         while (!q.empty()) {
24             ++visited;
25             int u = q.front();
26             q.pop();
27             for (int v: edges[u]) {
28                 --indeg[v];

```

```

29         if (indeg[v] == 0) {
30             q.push(v);
31         }
32     }
33 }
34
35 return visited == numCourses;
36 }
37 };

```

扩展部分：

岛屿最大面积

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int count;
5  int dir[4][2] = {0, 1, 1, 0, -1, 0, 0, -1}; // 四个方向
6  void dfs(vector<vector<int>>& grid, vector<vector<bool>>& visited, int x,
7  int y) {
8      for (int i = 0; i < 4; i++) {
9          int nextx = x + dir[i][0];
10         int nexty = y + dir[i][1];
11         if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
12         grid[0].size()) continue; // 越界了, 直接跳过
13         if (!visited[nextx][nexty] && grid[nextx][nexty] == 1) { // 没有访问过
14             的 同时 是陆地的
15             visited[nextx][nexty] = true;
16             count++;
17             dfs(grid, visited, nextx, nexty);
18         }
19     }
20 }
21
22 int main() {
23     int n, m;
24     cin >> n >> m;
25     vector<vector<int>> grid(n, vector<int>(m, 0));
26     for (int i = 0; i < n; i++) {
27         for (int j = 0; j < m; j++) {
28             cin >> grid[i][j];
29         }
30     }
31     vector<vector<bool>> visited(n, vector<bool>(m, false));
32     int result = 0;
33     for (int i = 0; i < n; i++) {
34         for (int j = 0; j < m; j++) {
35             if (!visited[i][j] && grid[i][j] == 1) {
36                 count = 1; // 因为dfs处理下一个节点, 所以这里遇到陆地了就先计数, dfs
37                 处理接下来的相邻陆地
38                 visited[i][j] = true;
39                 dfs(grid, visited, i, j); // 将与其链接的陆地都标记上 true

```

```

36         result = max(result, count);
37     }
38 }
39 }
40 cout << result << endl;
41
42 }
```

孤岛面积

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int dir[4][2] = {-1, 0, 0, -1, 1, 0, 0, 1}; // 保存四个方向
5  void dfs(vector<vector<int>>& grid, int x, int y) {
6      grid[x][y] = 0;
7      for (int i = 0; i < 4; i++) { // 向四个方向遍历
8          int nextx = x + dir[i][0];
9          int nexty = y + dir[i][1];
10         // 超过边界
11         if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
grid[0].size()) continue;
12         // 不符合条件, 不继续遍历
13         if (grid[nextx][nexty] == 0) continue;
14
15         dfs (grid, nextx, nexty);
16     }
17     return;
18 }
19
20 int main() {
21     int n, m;
22     cin >> n >> m;
23     vector<vector<int>> grid(n, vector<int>(m, 0));
24     for (int i = 0; i < n; i++) {
25         for (int j = 0; j < m; j++) {
26             cin >> grid[i][j];
27         }
28     }
29
30     // 从左侧边, 和右侧边 向中间遍历
31     for (int i = 0; i < n; i++) {
32         if (grid[i][0] == 1) dfs(grid, i, 0);
33         if (grid[i][m - 1] == 1) dfs(grid, i, m - 1);
34     }
35     // 从上边和下边 向中间遍历
36     for (int j = 0; j < m; j++) {
37         if (grid[0][j] == 1) dfs(grid, 0, j);
38         if (grid[n - 1][j] == 1) dfs(grid, n - 1, j);
39     }
40     int count = 0;
41     for (int i = 0; i < n; i++) {
42         for (int j = 0; j < m; j++) {
43             if (grid[i][j] == 1) count++;
44         }
45     }
```

```

45     }
46     cout << count << endl;
47 }

```

沉没孤岛

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int dir[4][2] = {-1, 0, 0, -1, 1, 0, 0, 1}; // 保存四个方向
5  void dfs(vector<vector<int>>& grid, int x, int y) {
6      grid[x][y] = 2;
7      for (int i = 0; i < 4; i++) { // 向四个方向遍历
8          int nextx = x + dir[i][0];
9          int nexty = y + dir[i][1];
10         // 超过边界
11         if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
grid[0].size()) continue;
12         // 不符合条件, 不继续遍历
13         if (grid[nextx][nexty] == 0 || grid[nextx][nexty] == 2) continue;
14         dfs (grid, nextx, nexty);
15     }
16     return;
17 }
18
19 int main() {
20     int n, m;
21     cin >> n >> m;
22     vector<vector<int>> grid(n, vector<int>(m, 0));
23     for (int i = 0; i < n; i++) {
24         for (int j = 0; j < m; j++) {
25             cin >> grid[i][j];
26         }
27     }
28
29     // 步骤一:
30     // 从左侧边, 和右侧边 向中间遍历
31     for (int i = 0; i < n; i++) {
32         if (grid[i][0] == 1) dfs(grid, i, 0);
33         if (grid[i][m - 1] == 1) dfs(grid, i, m - 1);
34     }
35
36     // 从上边和下边 向中间遍历
37     for (int j = 0; j < m; j++) {
38         if (grid[0][j] == 1) dfs(grid, 0, j);
39         if (grid[n - 1][j] == 1) dfs(grid, n - 1, j);
40     }
41     // 步骤二、步骤三
42     for (int i = 0; i < n; i++) {
43         for (int j = 0; j < m; j++) {
44             if (grid[i][j] == 1) grid[i][j] = 0;
45             if (grid[i][j] == 2) grid[i][j] = 1;
46         }
47     }
48     for (int i = 0; i < n; i++) {

```

```

49     for (int j = 0; j < m; j++) {
50         cout << grid[i][j] << " ";
51     }
52     cout << endl;
53 }
54 }

```

字符串接龙

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <unordered_set>
5  #include <unordered_map>
6  #include <queue>
7  using namespace std;
8  int main() {
9      string beginStr, endStr, str;
10     int n;
11     cin >> n;
12     unordered_set<string> strSet;
13     cin >> beginStr >> endStr;
14     for (int i = 0; i < n; i++) {
15         cin >> str;
16         strSet.insert(str);
17     }
18
19     // 记录strSet里的字符串是否被访问过，同时记录路径长度
20     unordered_map<string, int> visitMap; // <记录的字符串, 路径长度>
21
22     // 初始化队列
23     queue<string> que;
24     que.push(beginStr);
25
26     // 初始化visitMap
27     visitMap.insert(pair<string, int>(beginStr, 1));
28
29     while(!que.empty()) {
30         string word = que.front();
31         que.pop();
32         int path = visitMap[word]; // 这个字符串在路径中的长度
33
34         // 开始在这个str中，挨个字符去替换
35         for (int i = 0; i < word.size(); i++) {
36             string newWord = word; // 用一个新字符串替换str，因为每次要替换一个字符
37
38             // 遍历26的字母
39             for (int j = 0; j < 26; j++) {
40                 newWord[i] = j + 'a';
41                 if (newWord == endStr) { // 发现替换字母后，字符串与终点字符串相同
42                     cout << path + 1 << endl; // 找到了路径
43                     return 0;
44                 }
45                 // 字符串集合里出现了newWord，并且newWord没有被访问过
46                 if (strSet.find(newWord) != strSet.end())

```

```

47         && visitMap.find(newWord) == visitMap.end()) {
48             // 添加访问信息，并将新字符串放到队列中
49             visitMap.insert(pair<string, int>(newWord, path + 1));
50             que.push(newWord);
51         }
52     }
53 }
54 }
55
56 // 没找到输出0
57 cout << 0 << endl;
58
59 }

```

有向图的完全可表达性

```

1 // 写法一: dfs 处理当前访问的节点
2 #include <iostream>
3 #include <vector>
4 #include <list>
5 using namespace std;
6
7 void dfs(const vector<list<int>>& graph, int key, vector<bool>& visited) {
8     if (visited[key]) {
9         return;
10    }
11    visited[key] = true;
12    list<int> keys = graph[key];
13    for (int key : keys) {
14        // 深度优先搜索遍历
15        dfs(graph, key, visited);
16    }
17 }
18
19 int main() {
20     int n, m, s, t;
21     cin >> n >> m;
22
23     // 节点编号从1到n, 所以申请 n+1 这么大的数组
24     vector<list<int>> graph(n + 1); // 邻接表
25     while (m--) {
26         cin >> s >> t;
27         // 使用邻接表, 表示 s -> t 是相连的
28         graph[s].push_back(t);
29     }
30     vector<bool> visited(n + 1, false);
31     dfs(graph, 1, visited);
32     //检查是否都访问到了
33     for (int i = 1; i <= n; i++) {
34         if (visited[i] == false) {
35             cout << -1 << endl;
36             return 0;
37         }
38     }
39     cout << 1 << endl;

```

```
40 }
41
```

并查集基础模板

用于判断两个节点是否连接

```
1  int n = 1005; // n根据题目中节点数量而定，一般比节点数量大一点就好
2  vector<int> father = vector<int> (n, 0); // C++里的一种数组结构
3
4  // 并查集初始化
5  void init() {
6      for (int i = 0; i < n; ++i) {
7          father[i] = i;
8      }
9  }
10 // 并查集里寻根的过程
11 int find(int u) {
12     return u == father[u] ? u : father[u] = find(father[u]); // 路径压缩
13 }
14
15 // 判断 u 和 v是否找到同一个根
16 bool issame(int u, int v) {
17     u = find(u);
18     v = find(v);
19     return u == v;
20 }
21
22 // 将v->u 这条边加入并查集
23 void join(int u, int v) {
24     u = find(u); // 寻找u的根
25     v = find(v); // 寻找v的根
26     if (u == v) return ; // 如果发现根相同，则说明在一个集合，不用两个节点相连直接返回
27     father[v] = u;
28 }
```

寻找存在路径

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int n; // 节点数量
6  vector<int> father = vector<int> (101, 0); // 按照节点大小定义数组大小
7
8  // 并查集初始化
9  void init() {
10     for (int i = 1; i <= n; i++) father[i] = i;
11 }
12 // 并查集里寻根的过程
13 int find(int u) {
14     return u == father[u] ? u : father[u] = find(father[u]);
15 }
16
```

```

17 // 判断 u 和 v是否找到同一个根
18 bool issame(int u, int v) {
19     u = find(u);
20     v = find(v);
21     return u == v;
22 }
23
24 // 将v->u 这条边加入并查集
25 void join(int u, int v) {
26     u = find(u); // 寻找u的根
27     v = find(v); // 寻找v的根
28     if (u == v) return ; // 如果发现根相同，则说明在一个集合，不用两个节点相连直接返回
29     father[v] = u;
30 }
31
32 int main() {
33     int m, s, t, source, destination;
34     cin >> n >> m;
35     init();
36     while (m--) {
37         cin >> s >> t;
38         join(s, t);
39     }
40     cin >> source >> destination;
41     if (issame(source, destination)) cout << 1 << endl;
42     else cout << 0 << endl;
43 }

```

最小生成树

最小生成树就是所有结点的最小连通子图，即用最小的成本将所有节点链接到一起，常见的算法包括prim算法和kruskal算法

prim算法

prim算法是从节点的角度采用贪心的策略每次寻找距离最小生成树最近的节点并加入到最小生成树中。

prim算法核心就是三步，我称为**prim三部曲**，大家一定要熟悉这三步，代码相对会好些很多：

1. 第一步，选距离生成树最近节点
2. 第二步，最近节点加入生成树
3. 第三步，更新非生成树节点到生成树的距离（即更新minDist数组，**用来记录每一个节点距离最小生成树的最近距离**）

```

1  #include<iostream>
2  #include<vector>
3  #include <climits>
4
5  using namespace std;
6  int main() {
7      int v, e;
8      int x, y, k;
9      cin >> v >> e;
10     // 填一个默认最大值，题目描述val最大为10000

```

```

11 vector<vector<int>> grid(v + 1, vector<int>(v + 1, 10001));
12 while (e--) {
13     cin >> x >> y >> k;
14     // 因为是双向图，所以两个方向都要填上
15     grid[x][y] = k;
16     grid[y][x] = k;
17
18 }
19 // 所有节点到最小生成树的最小距离
20 vector<int> minDist(v + 1, 10001);
21
22 // 这个节点是否在树里
23 vector<bool> isInTree(v + 1, false);
24
25 // 我们只需要循环 n-1次，建立 n - 1条边，就可以把n个节点的图连在一起
26 for (int i = 1; i < v; i++) {
27
28     // 1、prim三部曲，第一步：选距离生成树最近节点
29     int cur = -1; // 选中哪个节点 加入最小生成树
30     int minVal = INT_MAX;
31     for (int j = 1; j <= v; j++) { // 1 - v，顶点编号，这里下标从1开始
32         // 选取最小生成树节点的条件：
33         // （1）不在最小生成树里
34         // （2）距离最小生成树最近的节点
35         if (!isInTree[j] && minDist[j] < minVal) {
36             minVal = minDist[j];
37             cur = j;
38         }
39     }
40     // 2、prim三部曲，第二步：最近节点（cur）加入生成树
41     isInTree[cur] = true;
42
43     // 3、prim三部曲，第三步：更新非生成树节点到生成树的距离（即更新minDist数组）
44     // cur节点加入之后， 最小生成树加入了新的节点，那么所有节点到 最小生成树的距离
    （即minDist数组）需要更新一下
45     // 由于cur节点是新加入到最小生成树，那么只需要关心与 cur 相连的 非生成树节点 的
    距离 是否比 原来 非生成树节点到生成树节点的距离更小了昵
46     for (int j = 1; j <= v; j++) {
47         // 更新的条件：
48         // （1）节点是 非生成树里的节点
49         // （2）与cur相连的某节点的权值 比 该某节点距离最小生成树的距离小
50         // 很多录友看到自己 就想不明白什么意思，其实就是 cur 是新加入 最小生成树的节
    点，那么 所有非生成树的节点距离生成树节点的最近距离 由于 cur的新加入，需要更新一下数据了
51         if (!isInTree[j] && grid[cur][j] < minDist[j]) {
52             minDist[j] = grid[cur][j];
53         }
54     }
55 }
56 // 统计结果
57 int result = 0;
58 for (int i = 2; i <= v; i++) { // 不计第一个顶点，因为统计的是边的权值，v个节点
    有 v-1条边
59     result += minDist[i];
60 }
61 cout << result << endl;
62

```

kruskal算法

kruskal的思路：

- 边的权值排序，因为要优先选最小的边加入到生成树里
- 遍历排序后的边
 - 如果边首尾的两个节点在同一个集合，说明如果连上这条边图中会出现环
 - 如果边首尾的两个节点不在同一个集合，加入到最小生成树，并把两个节点加入同一个集合

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  // l,r为 边两边的节点，val为边的数值
8  struct Edge {
9      int l, r, val;
10 };
11
12 // 节点数量
13 int n = 10001;
14 // 并查集标记节点关系的数组
15 vector<int> father(n, -1); // 节点编号是从1开始的，n要大一些
16
17 // 并查集初始化
18 void init() {
19     for (int i = 0; i < n; ++i) {
20         father[i] = i;
21     }
22 }
23
24 // 并查集的查找操作
25 int find(int u) {
26     return u == father[u] ? u : father[u] = find(father[u]); // 路径压缩
27 }
28
29 // 并查集的加入集合
30 void join(int u, int v) {
31     u = find(u); // 寻找u的根
32     v = find(v); // 寻找v的根
33     if (u == v) return ; // 如果发现根相同，则说明在一个集合，不用两个节点相连直接返回
34     father[v] = u;
35 }
36
37 int main() {
38
39     int v, e;
40     int v1, v2, val;
41     vector<Edge> edges;
42     int result_val = 0;
43     cin >> v >> e;

```

```

44     while (e--) {
45         cin >> v1 >> v2 >> val;
46         edges.push_back({v1, v2, val});
47     }
48
49     // 执行Kruskal算法
50     // 按边的权值对边进行从小到大排序
51     sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
52         return a.val < b.val;
53     });
54
55     // 并查集初始化
56     init();
57
58     // 从头开始遍历边
59     for (Edge edge : edges) {
60         // 并查集，搜出两个节点的祖先
61         int x = find(edge.l);
62         int y = find(edge.r);
63
64         // 如果祖先不同，则不在同一个集合
65         if (x != y) {
66             result_val += edge.val; // 这条边可以作为生成树的边
67             join(x, y); // 两个节点加入到同一个集合
68         }
69     }
70     cout << result_val << endl;
71     return 0;
72 }

```

拓扑排序

给出一个有向图，把这个有向图转成线性的排序就叫拓扑排序。拓扑排序也要检测这个有向图是否有环，即存在循环依赖的情况，因为这种情况是不能做线性排序的。所以拓扑排序也是图论中判断有向无环图的常用方法。

实现拓扑排序的算法有两种：卡恩算法（BFS）和DFS，一般来说我们只需要掌握 BFS（广度优先搜索）就可以了，清晰易懂

拓扑排序的过程，其实就两步：

1. 找到入度为0的节点，加入结果集
2. 将该节点从图中移除

循环以上两步，直到所有节点都在图中被移除了。

结果集的顺序，就是我们想要的拓扑排序顺序，如果我们发现结果集元素个数不等于图中节点个数，我们就可以认定图中一定有有向环！

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <unordered_map>
5  using namespace std;
6  int main() {
7      int m, n, s, t;

```

```

8      cin >> n >> m;
9      vector<int> inDegree(n, 0); // 记录每个文件的入度
10
11     unordered_map<int, vector<int>> umap; // 记录文件依赖关系
12     vector<int> result; // 记录结果
13
14     while (m--) {
15         // s->t, 先有s才能有t
16         cin >> s >> t;
17         inDegree[t]++; // t的入度加一
18         umap[s].push_back(t); // 记录s指向哪些文件
19     }
20     queue<int> que;
21     for (int i = 0; i < n; i++) {
22         // 入度为0的文件, 可以作为开头, 先加入队列
23         if (inDegree[i] == 0) que.push(i);
24         //cout << inDegree[i] << endl;
25     }
26     // int count = 0;
27     while (que.size()) {
28         int cur = que.front(); // 当前选中的文件
29         que.pop();
30         //count++;
31         result.push_back(cur);
32         vector<int> files = umap[cur]; //获取该文件指向的文件
33         if (files.size()) { // cur有后续文件
34             for (int i = 0; i < files.size(); i++) {
35                 inDegree[files[i]]--; // cur的指向的文件入度-1
36                 if(inDegree[files[i]] == 0) que.push(files[i]);
37             }
38         }
39     }
40     if (result.size() == n) {
41         for (int i = 0; i < n - 1; i++) cout << result[i] << " ";
42         cout << result[n - 1];
43     } else cout << -1 << endl;
44
45
46 }

```

dijkstra算法

dijkstra算法：在有权图（权值非负数）中求从起点到其他节点的最短路径算法。

需要注意两点：

- dijkstra 算法可以同时求 起点到所有节点的最短路径
- 权值不能为负数

dijkstra三部曲：

1. 第一步，选源点到哪个节点近且该节点未被访问过
2. 第二步，该最近节点被标记访问过
3. 第三步，更新非访问节点到源点的距离（即更新minDist数组，**minDist数组 用来记录 每一个节点距离源点的最小距离**）

```

1  #include <iostream>
2  #include <vector>
3  #include <climits>
4  using namespace std;
5  int main() {
6      int n, m, p1, p2, val;
7      cin >> n >> m;
8
9      vector<vector<int>> grid(n + 1, vector<int>(n + 1, INT_MAX));
10     for(int i = 0; i < m; i++){
11         cin >> p1 >> p2 >> val;
12         grid[p1][p2] = val;
13     }
14
15     int start = 1;
16     int end = n;
17
18     // 存储从源点到每个节点的最短距离
19     std::vector<int> minDist(n + 1, INT_MAX);
20
21     // 记录顶点是否被访问过
22     std::vector<bool> visited(n + 1, false);
23
24     minDist[start] = 0; // 起始点到自身的距离为0
25
26     for (int i = 1; i <= n; i++) { // 遍历所有节点
27
28         int minVal = INT_MAX;
29         int cur = 1;
30
31         // 1、选距离源点最近且未访问过的节点
32         for (int v = 1; v <= n; ++v) {
33             if (!visited[v] && minDist[v] < minVal) {
34                 minVal = minDist[v];
35                 cur = v;
36             }
37         }
38
39         visited[cur] = true; // 2、标记该节点已被访问
40
41         // 3、第三步，更新非访问节点到源点的距离（即更新minDist数组）
42         for (int v = 1; v <= n; v++) {
43             if (!visited[v] && grid[cur][v] != INT_MAX && minDist[cur] +
grid[cur][v] < minDist[v]) {
44                 minDist[v] = minDist[cur] + grid[cur][v];
45             }
46         }
47     }
48
49     if (minDist[end] == INT_MAX) cout << -1 << endl; // 不能到达终点
50     else cout << minDist[end] << endl; // 到达终点最短路径
51
52 }
53

```

prim是求 非访问节点到最小生成树的最小距离，而 **dijkstra**是求 非访问节点到源点的最小距离。

prim 更新 minDist数组的写法：

```
1 for (int j = 1; j <= v; j++) {
2     if (!isInTree[j] && grid[cur][j] < minDist[j]) {
3         minDist[j] = grid[cur][j];
4     }
5 }
```

因为 minDist表示 节点到最小生成树的最小距离，所以 新节点cur的加入，只需要使用 grid[cur][j]，grid[cur][j] 就表示 cur 加入生成树后，生成树到 节点j 的距离。

dijkstra 更新 minDist数组的写法：

```
1 for (int v = 1; v <= n; v++) {
2     if (!visited[v] && grid[cur][v] != INT_MAX && minDist[cur] + grid[cur][v]
3     < minDist[v]) {
4         minDist[v] = minDist[cur] + grid[cur][v];
5     }
6 }
```

因为 minDist表示 节点到源点的最小距离，所以 新节点 cur 的加入，需要使用 源点到cur的距离 (minDist[cur]) + cur 到 节点 v 的距离 (grid[cur][v])，才是 源点到节点v的距离。

Bellman_ford算法

用来解决带负值的最短路径消耗问题，**Bellman_ford算法的核心思想是 对所有边进行松弛n-1次操作 (n为节点数量)，从而求得目标最短路。**使用minDist数组来表达 起点到各个节点的最短距离。

围绕以下这句代码展开：

```
1 if (minDist[B] > minDist[A] + value) minDist[B] = minDist[A] + value
```

这句代码就是 Bellman_ford算法的核心操作。

以上代码也可以这么写：`minDist[B] = min(minDist[A] + value, minDist[B])`

如果大家看过代码随想录的动态规划章节，会发现 无论是背包问题还是子序列问题，这段代码（递推公式）出现频率非常高的。

其实 Bellman_ford算法 也是采用了动态规划的思想，即：将一个问题分解成多个决策阶段，通过状态之间的递归关系最后计算出全局最优解。**对所有边松弛一次，相当于计算 起点到达 与起点一条边相连的节点的最短距离**，起点 到达 所有节点的最短距离，因为所有节点与起点连接的边数最多也就是 n-1 条边。

```
1 #include <iostream>
2 #include <vector>
3 #include <list>
4 #include <climits>
5 using namespace std;
6
7 int main() {
8     int n, m, p1, p2, val;
9     cin >> n >> m;
10 }
```

```

11     vector<vector<int>> grid;
12
13     // 将所有边保存起来
14     for(int i = 0; i < m; i++){
15         cin >> p1 >> p2 >> val;
16         // p1 指向 p2, 权值为 val
17         grid.push_back({p1, p2, val});
18
19     }
20     int start = 1; // 起点
21     int end = n;   // 终点
22
23     vector<int> minDist(n + 1, INT_MAX);
24     minDist[start] = 0;
25     for (int i = 1; i < n; i++) { // 对所有边 松弛 n-1 次
26         for (vector<int> &side : grid) { // 每一次松弛, 都是对所有边进行松弛
27             int from = side[0]; // 边的出发点
28             int to = side[1]; // 边的到达点
29             int price = side[2]; // 边的权值
30             // 松弛操作
31             // minDist[from] != INT_MAX 防止从未计算过的节点出发
32             if (minDist[from] != INT_MAX && minDist[to] > minDist[from] +
price) {
33                 minDist[to] = minDist[from] + price;
34             }
35         }
36     }
37     if (minDist[end] == INT_MAX) cout << "unconnected" << endl; // 不能到达终
点
38     else cout << minDist[end] << endl; // 到达终点最短路径
39
40 }

```

判断负权回路：有负权回路的情况下，一直都会有更短的最短路，所以 松弛 第n次，minDist数组 也会发生改变。那么解决本题的核心思路，就是在 [kama94.城市间货物运输I](#) 的基础上，再多松弛一次，看 minDist数组是否发生变化。

单源有限最短路：经过有限个结点的最小消耗路径

Floyd算法

多源最短路，即 求多个起点到多个终点的多条最短路径。**Floyd 算法对边的权值正负没有要求，都可以处理。**结合动态规划， $grid[i][j][k] = m$ ，表示 **节点i 到 节点j 以[1...k] 集合中的一个节点为中间节点的最短距离为m**，

分两种情况：

1. 节点i 到 节点j 的最短路径经过节点k
2. 节点i 到 节点j 的最短路径不经过节点k

对于第一种情况， $grid[i][j][k] = grid[i][k][k - 1] + grid[k][j][k - 1]$

节点i 到 节点k 的最短距离 是不经过节点k，中间节点集合为[1...k-1]，所以表示为 $grid[i][k][k - 1]$

节点k 到 节点j 的最短距离 也是不经过节点k，中间节点集合为[1...k-1]，所以表示为 $grid[k][j][k - 1]$

第二种情况, `grid[i][j][k] = grid[i][j][k - 1]`

如果节点i 到 节点j的最短距离 不经过节点k, 那么 中间节点集合[1...k-1], 表示为 `grid[i][j][k - 1]`

因为我们是求最短路, 对于这两种情况自然是取最小值。

即: `grid[i][j][k] = min(grid[i][k][k - 1] + grid[k][j][k - 1], grid[i][j][k - 1])`

	适用图的大小	边的权值为负数	检测负权回路	有限节点最短路径	源点数	时间复杂度
Dijkstra 朴素版	稠密图	不可以	不可以	不可以	单源	$O(N^2)$ N为节点数量
Dijkstra 堆优化版	稀疏图	不可以	不可以	不可以	单源	$O(E \log E)$ E为边的数量
Bellman-Ford	稠密图	可以	可以	可以	单源	$O(N * E)$ N为节点数量, E为边的数量
SPFA (Bellman_ford 队列优化)	稀疏图	可以	可以	可以	单源	$O(K * N)$ K为不定值, 取决于图的稠密度
Floyd	稠密图	可以	可以	不可以	多源	$O(N^3)$ N为节点数量

其他 (技巧类)

只出现一次的数字

使用异或技巧, 由于交换律, 只要有一个数出现一次以上, 该部分就为0, 0异或一个数等于这个数字本身

```

1 class Solution {
2 public:
3     int singleNumber(vector<int>& nums) {
4         int ret = 0;
5         for (auto e: nums) ret ^= e;
6         return ret;
7     }
8 };

```

多数元素

一个数如果个数超过n的一半, 那么排序后数组, n/2位置的数一定是该数

```

1 class Solution {
2 public:
3     int majorityElement(vector<int>& nums) {
4         sort(nums.begin(), nums.end());
5         return nums[nums.size() / 2];
6     }
7 };

```

颜色分类

```

1 class Solution {
2 public:
3     void sortColors(vector<int>& nums) {
4         int n = nums.size();

```

```

5     int p0 = 0, p1 = 0;
6     for (int i = 0; i < n; ++i) {
7         if (nums[i] == 1) {
8             swap(nums[i], nums[p1]);
9             ++p1;
10        } else if (nums[i] == 0) {
11            swap(nums[i], nums[p0]);
12            if (p0 < p1) {
13                swap(nums[i], nums[p1]);
14            }
15            ++p0;
16            ++p1;
17        }
18    }
19 }
20 };

```

下一个排列

```

1  class Solution {
2  public:
3      void nextPermutation(vector<int>& nums) {
4          int i = nums.size() - 2;
5          while (i >= 0 && nums[i] >= nums[i + 1]) {
6              i--;
7          }
8          if (i >= 0) {
9              int j = nums.size() - 1;
10             while (j >= 0 && nums[i] >= nums[j]) {
11                 j--;
12             }
13             swap(nums[i], nums[j]);
14         }
15         reverse(nums.begin() + i + 1, nums.end());
16     }
17 };

```

寻找重复数

```

1  class Solution {
2  public:
3      int findDuplicate(vector<int>& nums) {
4          int slow = 0, fast = 0;
5          do {
6              slow = nums[slow];
7              fast = nums[nums[fast]];
8          } while (slow != fast);
9          slow = 0;
10         while (slow != fast) {
11             slow = nums[slow];
12             fast = nums[fast];
13         }
14         return slow;
15     }

```

```
16 | };
```

未解决

随机链表的复制

```
1  class Solution {
2  public:
3      unordered_map<Node*, Node*> cachedNode;
4
5      Node* copyRandomList(Node* head) {
6          if (head == nullptr) {
7              return nullptr;
8          }
9          if (!cachedNode.count(head)) {
10             Node* headNew = new Node(head->val);
11             cachedNode[head] = headNew;
12             headNew->next = copyRandomList(head->next);
13             headNew->random = copyRandomList(head->random);
14         }
15         return cachedNode[head];
16     }
17 };
```

LRU缓存

```
1  struct DLinkedNode {
2      int key, value;
3      DLinkedNode* prev;
4      DLinkedNode* next;
5      DLinkedNode(): key(0), value(0), prev(nullptr), next(nullptr) {}
6      DLinkedNode(int _key, int _value): key(_key), value(_value),
7      prev(nullptr), next(nullptr) {}
8  };
9
10 class LRUCache {
11 private:
12     unordered_map<int, DLinkedNode*> cache;
13     DLinkedNode* head;
14     DLinkedNode* tail;
15     int size;
16     int capacity;
17 public:
18     LRUCache(int _capacity): capacity(_capacity), size(0) {
19         // 使用伪头部和伪尾部节点
20         head = new DLinkedNode();
21         tail = new DLinkedNode();
22         head->next = tail;
23         tail->prev = head;
24     }
25 }
```

```

26     int get(int key) {
27         if (!cache.count(key)) {
28             return -1;
29         }
30         // 如果 key 存在, 先通过哈希表定位, 再移到头部
31         DListNode* node = cache[key];
32         moveToHead(node);
33         return node->value;
34     }
35
36     void put(int key, int value) {
37         if (!cache.count(key)) {
38             // 如果 key 不存在, 创建一个新的节点
39             DListNode* node = new DListNode(key, value);
40             // 添加进哈希表
41             cache[key] = node;
42             // 添加至双向链表的头部
43             addToHead(node);
44             ++size;
45             if (size > capacity) {
46                 // 如果超出容量, 删除双向链表的尾部节点
47                 DListNode* removed = removeTail();
48                 // 删除哈希表中对应的项
49                 cache.erase(removed->key);
50                 // 防止内存泄漏
51                 delete removed;
52                 --size;
53             }
54         }
55         else {
56             // 如果 key 存在, 先通过哈希表定位, 再修改 value, 并移到头部
57             DListNode* node = cache[key];
58             node->value = value;
59             moveToHead(node);
60         }
61     }
62
63     void addToHead(DListNode* node) {
64         node->prev = head;
65         node->next = head->next;
66         head->next->prev = node;
67         head->next = node;
68     }
69
70     void removeNode(DListNode* node) {
71         node->prev->next = node->next;
72         node->next->prev = node->prev;
73     }
74
75     void moveToHead(DListNode* node) {
76         removeNode(node);
77         addToHead(node);
78     }
79
80     DListNode* removeTail() {
81         DListNode* node = tail->prev;

```

```

82         removeNode(node);
83         return node;
84     }
85 };

```

实现Tire（前缀树）

```

1  class Trie {
2  private:
3      vector<Trie*> children;
4      bool isEnd;
5
6      Trie* searchPrefix(string prefix) {
7          Trie* node = this;
8          for (char ch : prefix) {
9              ch -= 'a';
10             if (node->children[ch] == nullptr) {
11                 return nullptr;
12             }
13             node = node->children[ch];
14         }
15         return node;
16     }
17
18 public:
19     Trie() : children(26), isEnd(false) {}
20
21     void insert(string word) {
22         Trie* node = this;
23         for (char ch : word) {
24             ch -= 'a';
25             if (node->children[ch] == nullptr) {
26                 node->children[ch] = new Trie();
27             }
28             node = node->children[ch];
29         }
30         node->isEnd = true;
31     }
32
33     bool search(string word) {
34         Trie* node = this->searchPrefix(word);
35         return node != nullptr && node->isEnd;
36     }
37
38     bool startswith(string prefix) {
39         return this->searchPrefix(prefix) != nullptr;
40     }
41 };

```

N皇后

```

1  class Solution {
2  private:
3      vector<vector<string>> result;

```

```

4 // n 为输入的棋盘大小
5 // row 是当前递归到棋盘的第几行了
6 void backtracking(int n, int row, vector<string>& chessboard) {
7     if (row == n) {
8         result.push_back(chessboard);
9         return;
10    }
11    for (int col = 0; col < n; col++) {
12        if (isValid(row, col, chessboard, n)) { // 验证合法就可以放
13            chessboard[row][col] = 'Q'; // 放置皇后
14            backtracking(n, row + 1, chessboard);
15            chessboard[row][col] = '.'; // 回溯，撤销皇后
16        }
17    }
18 }
19 bool isValid(int row, int col, vector<string>& chessboard, int n) {
20     // 检查列
21     for (int i = 0; i < row; i++) { // 这是一个剪枝
22         if (chessboard[i][col] == 'Q') {
23             return false;
24         }
25     }
26     // 检查 45度角是否有皇后
27     for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
28         if (chessboard[i][j] == 'Q') {
29             return false;
30         }
31     }
32     // 检查 135度角是否有皇后
33     for (int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
34         if (chessboard[i][j] == 'Q') {
35             return false;
36         }
37     }
38     return true;
39 }
40 public:
41     vector<vector<string>> solveNQueens(int n) {
42         result.clear();
43         std::vector<std::string> chessboard(n, std::string(n, '.'));
44         backtracking(n, 0, chessboard);
45         return result;
46     }
47 };

```

寻找两个正序数组的中位数

```

1 class Solution {
2 public:
3     int getKthElement(const vector<int>& nums1, const vector<int>& nums2,
4 int k) {
5
6         int m = nums1.size();
7         int n = nums2.size();
8         int index1 = 0, index2 = 0;
9
10    }
11 };

```

```

8
9     while (true) {
10         // 边界情况
11         if (index1 == m) {
12             return nums2[index2 + k - 1];
13         }
14         if (index2 == n) {
15             return nums1[index1 + k - 1];
16         }
17         if (k == 1) {
18             return min(nums1[index1], nums2[index2]);
19         }
20
21         // 正常情况
22         int newIndex1 = min(index1 + k / 2 - 1, m - 1);
23         int newIndex2 = min(index2 + k / 2 - 1, n - 1);
24         int pivot1 = nums1[newIndex1];
25         int pivot2 = nums2[newIndex2];
26         if (pivot1 <= pivot2) {
27             k -= newIndex1 - index1 + 1;
28             index1 = newIndex1 + 1;
29         }
30         else {
31             k -= newIndex2 - index2 + 1;
32             index2 = newIndex2 + 1;
33         }
34     }
35 }
36
37 double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
38     int totalLength = nums1.size() + nums2.size();
39     if (totalLength % 2 == 1) {
40         return getKthElement(nums1, nums2, (totalLength + 1) / 2);
41     }
42     else {
43         return (getKthElement(nums1, nums2, totalLength / 2) +
44 getKthElement(nums1, nums2, totalLength / 2 + 1)) / 2.0;
45     }
46 };

```

数据流的中位数

```

1 class MedianFinder {
2 public:
3     priority_queue<int, vector<int>, less<int>> queMin;
4     priority_queue<int, vector<int>, greater<int>> queMax;
5
6     MedianFinder() {}
7
8     void addNum(int num) {
9         if (queMin.empty() || num <= queMin.top()) {
10             queMin.push(num);
11             if (queMax.size() + 1 < queMin.size()) {
12                 queMax.push(queMin.top());

```

```
13         queMin.pop();
14     }
15 } else {
16     queMax.push(num);
17     if (queMax.size() > queMin.size()) {
18         queMin.push(queMax.top());
19         queMax.pop();
20     }
21 }
22 }
23
24 double findMedian() {
25     if (queMin.size() > queMax.size()) {
26         return queMin.top();
27     }
28     return (queMin.top() + queMax.top()) / 2.0;
29 }
30 };
```